

Wasp: Succinct Non-Interactive Zero-Knowledge Proofs from VOLE

Zhanpeng Guo*

Ant Group
Beijing, China
China Telecom Quantum Information
Technology Group Co., Ltd
Hefei, China
zhanp.guo@gmail.com

Chenkai Weng
Arizona State University
Tempe, USA
chenkai.weng@asu.edu

Zhelei Zhou

Ant Group
Hangzhou, China
zhouzhelei.zzl@antgroup.com

Cheng Hong
Ant Group
Beijing, China
vince.hc@antgroup.com

Yun Li†

Ant Group
Beijing, China
liyun24@antgroup.com

Tao Wei
Ant Group
Hangzhou, China
lenx.wei@antgroup.com

Abstract

Zero-knowledge proofs (ZKPs) based on vector oblivious linear evaluation (VOLE) excel in prover efficiency but typically require linear communication and verification. Antman (Weng et al., CCS '22) introduced information-theoretic polynomial authentication codes (IT-PACs) to achieve sublinear communication: $O(B + C)$ for SIMD (single-instruction-multiple-data) circuits and $O(B^3 + C)$ for general circuits, where $N = B \cdot C$ is the total circuit size and B, C are batch size and subcircuit size, respectively. Antman++ (Bui et al., J. Cryptol. '25) further reduced the general-case communication to $O(B + C)$. However, these protocols remain interactive and cannot be made non-interactive via traditional techniques like Fiat-Shamir, due to limited functionalities of IT-PACs; also, the verifier of Antman++ is not succinct for processing $N \times N$ public matrices.

In this work, we present a succinct non-interactive ZKP system Wasp. Specifically, (1) we enhance the IT-PAC primitive to a fully functional polynomial commitment scheme (PCS) with the support of generic evaluation openings. With this PCS, we construct Wasp^S, a non-interactive ZKP for SIMD circuits based on Antman; also, we build Wasp^G, a general zkSNARK with constant verifier time and proof size based on the Plonkish constraint system. (2) We optimize the SIMD-to-general compiler from Antman++ by exploiting sparse representation of matrices and extending preprocessing techniques to the SIMD setting, and achieve a sublinear verifier. All our protocols achieve non-interactivity in the VOLE-hybrid model (i.e., given preprocessed VOLE correlations). Experiments show the non-interactive verifier of our SIMD zkSNARK Wasp^S is 1 ~ 2 orders of magnitude faster than the interactive one of Antman; when compiled with our compiler, the general-case verifier is 2 ~ 3 orders of magnitude faster than the one in Antman++. Our general

zkSNARK Wasp^G has a 1 ~ 2 orders of magnitude faster prover than pairing-based, coding-based and lattice-based zkSNARKs, with less than 1 ms verifier time and 122 KB proof size; compared to the non-succinct VOLE-based ZKP, Wasp^G is 3 ~ 22× slower in proving but can be 4 orders of magnitude faster in verification with 3 orders of magnitude smaller proof size.

CCS Concepts

• Security and privacy → Cryptography.

Keywords

VOLE; PCS; zkSNARKs

ACM Reference Format:

Zhanpeng Guo, Zhelei Zhou, Yun Li, Chenkai Weng, Cheng Hong, and Tao Wei. 2026. Wasp: Succinct Non-Interactive Zero-Knowledge Proofs from VOLE. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 24 pages. <https://doi.org/10.1145/3830454.3846743>

1 Introduction

Zero-knowledge proofs (ZKPs) enable a party to show the truth of an NP statement without revealing any information other than the truth of the statement itself. Recent decades have witnessed great advancements in ZKPs since they were first introduced by Goldwasser et al. in 1985 [22]. Among diverse types of ZKPs, zero-knowledge succinct¹ non-interactive arguments of knowledge (zkSNARKs) stand out due to their sublinear proof sizes and verifier time, and are especially prevalent in real-world deployments (such as Plonk [21] and Marlin [15]).

Vector oblivious linear evaluation (VOLE)-based ZK protocols [6, 10, 18, 40, 41, 43] are an emerging class of ZKPs that feature high prover efficiency. They are typically built on the learning parity with noise (LPN) assumption, and are a type of designated-verifier non-interactive ZK (DV-NIZK) protocol given VOLE correlations generated by a preprocessing phase. They adopt a gate-by-gate proving paradigm, and their provers can be much more efficient than

*Work done during an internship at Ant Group.

†Yun Li is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. CCS '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2871-6/2026/11

<https://doi.org/10.1145/3830454.3846743>

¹"Succinct" means proof size and verification time are sublinear in circuit size.

those of traditional zkSNARKs. The recent VOLE-in-the-Head [5] technique can eliminate the designated-verifier preprocessing phase and turn these protocols into publicly verifiable ones. However, they typically entail linear proof size and verification time.

Antman [42] first achieves sublinear communication cost among VOLE-based ZKPs, for both SIMD (single instruction multiple data) circuits and generic circuits. For a SIMD circuit consisting of B instances of a subcircuit of size C (and thus the total circuit size is $N = B \cdot C$), it is natural to view their wire values as a matrix: each column describes one execution, while each row contains the B values carried by the same wire across all executions. Instead of authenticating these B values separately, Antman encodes each row as a polynomial and authenticates it with a new notion called information-theoretic polynomial authentication code (IT-PAC). This yields $O(B + C)$ communication, minimized to $O(N^{1/2})$ when $B = C = N^{1/2}$. To handle a general circuit, Antman reorganizes the general circuit into SIMD subcircuits with padded dummy gates, which are then proved using its SIMD ZKP. Note that correct SIMD executions alone do not guarantee a valid general-circuit execution; it is also necessary to ensure the consistency of wire connections among these SIMD subcircuits w.r.t. the original circuit. Antman thus separately performs a wire consistency check, and this additionally incurs $O(B^3)$ communication cost, resulting in $O(B^3 + C)$ total communication, or $O(N^{3/4})$ with the optimal choice $B = N^{1/4}$.

Antman is a pioneering work in the VOLE-based ZKP family for achieving sublinear communication cost. However, we notice that it cannot be made non-interactive as previous VOLE-based ZKPs using folklore techniques (e.g., the Fiat-Shamir transformation [19]). This arises from its IT-PAC mechanism. At a high level, an IT-PAC is a designated-verifier commitment to a polynomial f : the prover holds a secret f , and authenticates it via a hidden evaluation of f at a secret point Λ , i.e., $f(\Lambda)$, using the additively homomorphic encryption (AHE) and VOLE primitives. Informally, binding holds as the prover does not know Λ , and hiding holds as $f(\Lambda)$ is masked when authenticating it using VOLE. However, unlike a fully functional polynomial commitment scheme (PCS), an IT-PAC lacks the capability to open the committed polynomial at any fresh verifier-chosen point. Antman's multiplication check therefore requires the verifier to reveal Λ after the polynomials have been authenticated to enable the prover evaluating them at Λ , and then complete the check. This online message cannot be removed by a direct Fiat-Shamir transformation [19], since Fiat-Shamir can only derive a fresh public challenge from the transcript, whereas Λ must already be fixed and kept secret when the IT-PACs are generated. Once revealed, Λ must also be refreshed before authenticating another batch of computations, and additional consistency checks between the old and refreshed IT-PACs may be required.

A follow-up work [12] proposed a compiler from SIMD to general ZKP, and built Antman++ by applying the compiler to the SIMD ZKP of Antman. It revisits the wire consistency check problem in Antman. To ensure the witnesses are consistent with the wire connections specified by the circuit, it collects all wire values in a global vector and uses three public sparse matrices (that describe the circuit structure) to select the left input, right input, and output values of all multiplication gates. Wire consistency is then reduced to three matrix-vector product equalities. It then compresses

each resulting matrix-vector product equality into an inner-product equality using a random linear combination, and proves it with the underlying SIMD ZKP. This compiler reduces the wire consistency communication from $O(B^3)$ to $O(B)$ and achieves an optimal total communication of $O(N^{1/2})$. However, its verifier has to explicitly process the circuit-size public matrices, which takes $O(N)$ time and thus is not succinct. Another recent work [31] explored multivariate sumcheck-based interactive proofs in the VOLE context. It achieves linear prover time. Nevertheless, the proof size and verifier time are $O(d \log N + d^2)$, where d is the depth of the circuit, and the round complexity is $O(d \log N)$ rather than constant.

1.1 Our Contributions

In this work, we present the Wasp ZKP system, consisting of three ZK protocols: Wasp^S, Wasp^G and Wasp^C to mitigate the above issues.² We outline our contributions below.

A New PCS from VOLE. We extend IT-PAC to a fully functional PCS, with the support of opening the committed polynomials at arbitrary points. The core technique is utilizing the polynomial divisibility property (as in the KZG10 PCS [28]) to create a witness polynomial; by committing the witness polynomial with an IT-PAC, the correctness of evaluations can be checked with VOLE techniques. It enables modular design of ZKPs by combining a PIOP (polynomial interactive oracle proof) system with a PCS, and may be of independent interest.

The complexities and properties of our PCS greatly resemble KZG10, i.e., the evaluation proof has $O(1)$ size and verification time, and is friendly for batching. Besides, as we use VOLE and AHE as the underlying cryptographic tools rather than the more costly pairing and group operations as in KZG10, our PCS is more efficient in committing and proving. Based on this, we construct Wasp^S and Wasp^G.

- Wasp^S targets SIMD circuits, built upon the SIMD ZKP from Antman. By adapting the multiplication check of Antman to a PIOP and combining it with our PCS, we can achieve a non-interactive verifier with higher concrete efficiency.
- Wasp^G is a zkSNARK for general circuits, constructed by combining our new PCS with the PIOP system from Plonk [21]. Consequently, it achieves constant proof size and verification time³, which, to the best of our knowledge, is the first VOLE-based ZKP achieving this property.

An Optimized SIMD-to-General ZK Protocol. We improve the SIMD-to-general ZK compiler in Antman++ [12] by designing an optimized wire consistency check protocol with a succinct verifier. The core idea is to reorganize the wire consistency relation as a matrix-vector multiplication in the SIMD setting, and preprocess the public matrix with a sparse representation. By extending existing preprocessing techniques [15, 16] to the SIMD setting, we can achieve an $O(\sqrt{N})$ verifier. Based on this, we construct Wasp^C.

- Wasp^C is a general ZKP obtained by compiling Wasp^S with our optimized compiler. Compared to Antman++, it can reduce

²"S" for SIMD, "G" for general, and "C" for compiler.

³Following SNARK literature, we don't count the $O(\log N)$ field operations (which are cheap) when saying verifier cost is "constant".

Table 1: Comparisons of different ZKPs. ($\mathbb{F}, \mathbb{P}, \mathbb{G}$: field, pairing, group operations. $\mathbb{A}_E, \mathbb{A}_D, \mathbb{L}_E, \mathbb{L}_D$: AHE encryption and decryption, lattice-based encryption and decryption operations. $\mathbb{H}$: hash operation. $|\cdot|$: the size of the corresponding element (for $\mathbb{F}, \mathbb{P}, \mathbb{G}$) or ciphertext (for $\mathbb{A}$ and $\mathbb{L}$). N denotes the total size of the SIMD or general circuit. For SIMD circuits, $N = BC$ where B denotes the batch size and C denotes the size of a SIMD subcircuit. Circ.: Circuit, Assump.: Assumption, Gen.: General, PQ.: Plausibly Post-Quantum Secure, PV.: Publicly Verifiable, Uni.: Universal, and “/” denotes transparent setup.)

ZKP	Circ. Type	Assump.	NIZK?	PQ.?	PV.?	Uni. Setup?	$\mathcal{P}$ Time	$\mathcal{V}$ Time	Proof Size [†]
Antman	SIMD	LPN, RLWE	$\times$	$\checkmark$	$\times$	$\checkmark$	$O(N \log B) \mathbb{F} + O(N) \mathbb{A}_E$	$O(C) \mathbb{A}_D + O(B) \mathbb{F}$	$O(C) \mathbb{A} + O(B) \mathbb{F} $
Wasp ^S			$\checkmark$	$\checkmark$	$\times$	$\checkmark$	$O(N \log B) \mathbb{F} + O(N) \mathbb{A}_E$	$O(C) \mathbb{A}_D + O(C) \mathbb{F}$	$O(C) \mathbb{A} + O(C) \mathbb{F} $
Antman*	SIMD-to-Gen.	LPN, RLWE	$\times$	$\checkmark$	$\times$	$\checkmark$	$O(N \log B) \mathbb{F} + O(N) \mathbb{A}_E$	$O(C) \mathbb{A}_D + O(B^3) \mathbb{F}$	$O(C) \mathbb{A} + O(B^3) \mathbb{F} $
Antman++			$\times$	$\checkmark$	$\times$	$\checkmark$	$O(N \log B) \mathbb{F} + O(N) \mathbb{A}_E$	$O(C) \mathbb{A}_D + O(N) \mathbb{F}$	$O(C) \mathbb{A} + O(B) \mathbb{F} $
Wasp ^C			$\checkmark$	$\checkmark$	$\times$	$\checkmark$	$O(N \log B) \mathbb{F} + O(N) \mathbb{A}_E$	$O(C) \mathbb{A}_D + O(\log B + C) \mathbb{F}$	$O(C) \mathbb{A} + O(C) \mathbb{F} $
Quicksilver	Gen.	LPN	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	$O(N) \mathbb{F}$	$O(N) \mathbb{F}$	$O(N) \mathbb{F} $
Plonk [21]		Pairing	$\checkmark$	$\times$	$\checkmark$	$\checkmark$	$O(N \log N) \mathbb{F} + O(N) \mathbb{G}$	$O(1) \mathbb{P} + O(\log N) \mathbb{F}$	$O(1) \mathbb{G} + O(1) \mathbb{F} $
Luna [38]		MLWE	$\checkmark$	$\checkmark$	$\times$	$\times$	$O(N \log N) \mathbb{F} + O(N) \mathbb{L}_E$	$O(1) \mathbb{L}_D + O(1) \mathbb{F}$	$O(1) \mathbb{L} $
WHIR [‡] [4]		Hash	$\checkmark$	$\checkmark$	$\checkmark$	/	$O(N \log N) \mathbb{F} + O(N) \mathbb{H}$	$O(\log N) \mathbb{F} + O(\log N) \mathbb{H}$	$O(\log N) \mathbb{F} + O(\log N) \mathbb{H}$
Wasp ^G		LPN, RLWE	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	$O(N \log N) \mathbb{F} + O(N) \mathbb{A}_E$	$O(1) \mathbb{A}_D + O(\log N) \mathbb{F}$	$O(1) \mathbb{A} + O(1) \mathbb{F} $

* This refers to the general ZKP compiled from the SIMD ZKP of Antman.

[†] Antman, Antman++ and Wasp^C achieves sublinear proof size $O(\sqrt{N})$ with $B = \sqrt{N}$; the general ZKP of Antman has $O(N^{3/4})$ proof size with $B = N^{1/4}$.

[‡] WHIR [4] is a PCS. Here we refer to the zkSNARK from [4] which is based on the WHIR PCS.

the $O(N)$ verifier time to $O(\sqrt{N})$ and meanwhile inherit the non-interaction property from Wasp^S.

- Differences between Wasp^G and Wasp^C: Although both target general circuits, Wasp^G features constant verifier time and proof size, while Wasp^C has $O(\sqrt{N})$ complexity. However, Wasp^C is more friendly to SIMD executions, and may be more favourable in multi-threading or parallelizable settings.

We give a comparison of our ZKPs with related works in Table 1. All our protocols and the other VOLE-based ZKPs listed in this table are in the VOLE-hybrid model, and non-interactivity refers to the online phase given prepared VOLE correlations. Theoretically, the improvements of Wasp^S on Antman mainly lie in achieving non-interaction, and Wasp^C additionally achieves a succinct verifier over Antman++. All of our protocols are non-interactive and succinct, as well as designated-verifier and plausibly post-quantum secure (given the plausible post-quantum instantiations of VOLE [44] and AHE [11]). The representative VOLE-based ZKP, Quicksilver [43], lacks succinctness compared with ours. The pairing-based zkSNARK Plonk [21] is publicly verifiable, but is not post-quantum secure. The SOTA lattice-based zkSNARK Luna [38] shares the same designated-verifier and plausible post-quantum properties as ours, but it requires circuit-specific setup rather than a universal one, and also would yield a much larger common reference string (CRS). The SOTA coding and hash-based zkSNARK from WHIR [4] is publicly verifiable and plausibly post-quantum secure, but its verifier time and proof size grow logarithmically with circuit size.

Implementation and Evaluation. We fully implemented and evaluated our new PCS and the Wasp system. (1) Our PCS can be 312×, 6.7× faster in committing, and 174×, 19.5× faster in evaluation opening than the pairing-based PCS KZG10 [28] and coding-based PCS WHIR [4]. The verifier time of our PCS is merely 0.3 ms,

with a proof size of 121 KB. Compared with the lattice-based PCS Greyhound, ours may be 1 ~ 2.34× and 1.6 ~ 5× slower in committing and evaluation opening, but can be 2,034× faster in verification. (2) Compared to the interactive SIMD ZKP of Antman, our non-interactive Wasp^S has a 31.14 ~ 120× faster verifier and a 1.38 ~ 1.46× faster prover, with slightly larger communication cost. (3) Compared to the SIMD-to-general ZKP Antman++, our Wasp^C shows a 55 ~ 3,384× faster verifier time, with 1.1 ~ 1.6× slower prover time. (4) Compared to the well-known pairing-based zkSNARK Plonk [21], the lattice-based zkSNARK ISW21 [27], and the coding-based zkSNARK DeepFold-HP (obtained by combining the DeepFold PCS with HyperPlonk [13] PIOP system), Wasp^G can be 241×, 18.4× and 8.4× faster in proving, respectively. Wasp^G has a constant verifier time at about 0.8 ms, which is slightly slower than the 0.5 ms of ISW21 but can be one order of magnitude faster than Plonk and DeepFold-HP. The proof size of Wasp^G is 122 KB. Compared to the linear-time Quicksilver, Wasp^G’s prover is about 3 ~ 22× slower; however, the succinct verifier of Wasp^G can be 4 orders of magnitude faster, and the proof size can be 3 orders of magnitude smaller at circuit size 2^{26} .

1.2 Related Works

VOLE-based ZKPs. This line of work was initiated by Boyle et al. [10], who introduced the first VOLE-based ZKP based on the LPN assumption. Subsequent works significantly improved upon this foundation. Wolverine [40], LPZK [18], LPZKv2 [17], Quicksilver [43], and Mac’n’Cheese [6] employ the authenticated multiplication triples to prove the multiplication gates for both arithmetic and boolean circuits. The SOTA work JesseQ [33] further reduces the online prover cost, with a relatively heavy preprocessing phase. All

these protocols feature linear prover complexity and are highly efficient, but they incur linear proof size and verification time. Antman is the first VOLE-based ZKP with sublinear communication cost, but lacks the non-interaction property; while Antman++ further improves the communication cost, its verifier time is linear. Lin et al. [31] also achieved sublinear costs; however, its higher round complexity $O(d \log N)$ would incur more efficiency loss when applying the Fiat-Shamir transformation over small fields, considering round-by-round soundness. The VOLE-in-the-Head (VOLE-itH) technique [5] is an orthogonal line that compiles a broad class of designated-verifier VOLE-based ZKPs into publicly verifiable ones. It enables the prover to first commit to the randomness underlying the VOLE correlations before receiving a public challenge, and subsequently opens all but one branch to allow any verifier to reconstruct and check the verifier-side correlations. Applying the Fiat-Shamir transformation then yields publicly verifiable proofs in the random-oracle model. This approach largely preserves the lightweight linear prover cost of VOLE-based ZKP. However, it cannot provide succinctness if the underlying VOLE-based ZKP is not succinct, and it typically retains the linear proof size and verification time of VOLE-based ZKPs.

Our work continues the VOLE-based line of research, and can achieve non-interaction, constant proof size and verification time for the first time in the VOLE-based ZKP family. Moreover, our protocols are compatible with the VOLE-itH technique. It can help to remove the preprocessing phase for generating VOLE correlations.⁴

Lattice-based zkSNARKs. Boneh et al. [27] proposed a general template for building lattice-based SNARKs by combining a linear PCP (probabilistically checkable proof) [24] with a linear-only encryption scheme. Given this, Ishai et al. [27] proposed a DV-zkSNARK with constant proof size and verifier time. However, due to the reliance on linear PCP, it requires an expensive circuit-dependent setup to generate a long CRS (e.g., it takes 37 minutes to generate a CRS of size 5.3 GB for a circuit of size 2^{20}). Luna [38] improved upon this construction with a shorter proof size, using a new candidate linear-only homomorphic encryption scheme called Module Half-GSW. However, the CRS size remains large.

Another line of research focuses on lattice-based PCS [1, 26, 34, 39]. Wee et al. [39] introduced a base folding technique for vector commitments, extended to polynomial commitments via Lagrange interpolation, achieving logarithmic proof size and verification time under SIS. However, the construction relies on relaxed soundness. SLAP [1] addressed this limitation through a refined polynomial decomposition that achieves standard soundness under Module-SIS. Greyhound [34] focuses on prover efficiency through FFT-based lattice arithmetic and reduces prover complexity to $O(N \log N)$. However, the proof size remains $O(\log N)$ rather than constant.

Coding-based zkSNARKs. Error-correction codes and hash offer an alternative way to construct plausible post-quantum zkSNARKs. Ligo [2] introduced the first code-based argument system by encoding polynomial evaluations as codewords and leveraging the proximity testing of Reed-Solomon codes. It achieves sublinear proof size $O(\sqrt{N})$, relying solely on collision-resistant hash

functions. The following work Brakedown [23] achieves linear-time prover cost through a linear-time encodable code, and then BaseFold [45] introduced a recursive folding procedure on codewords and reduced verification complexity to polylogarithmic while maintaining a linear-time prover. STIR [3] reduces the number of query rounds and improves both proof size and verification time with minimal prover overhead. However, the verification time remains $O(\log^2 N)$. The subsequent work WHIR [4] further improves the proximity testing, achieving logarithmic verification time. The SOTA work DeepFold [25] constructs an efficient multilinear polynomial commitment from Reed-Solomon codes with optimized folding procedures, and improves concrete efficiencies.

2 Preliminaries

Notation. Let $\mathbb{F}$ denote a finite field. We write $r \leftarrow \mathbb{F}$ for sampling r uniformly at random from $\mathbb{F}$. Vectors are denoted by arrows, e.g., $\vec{a}$, and $a[i]$ denotes the i -th component of $\vec{a}$; matrices are denoted by bold capital letters, e.g., $\mathbf{M}$. Index begins at zero, and $[n]$ denotes the set $\{0, \dots, n-1\}$ for $n \in \mathbb{N}$. We use $f \in \mathbb{F}_{\leq d}[X]$ to denote a polynomial over $\mathbb{F}$ with degree $\deg(f) \leq d$. We use κ and λ to denote the computational and statistical security parameter, respectively. The notation $\text{negl}(\cdot)$ denotes a negligible function, and $\Pr[\cdot]$ denotes probability. An algorithm is PPT if it is probabilistic and runs in time polynomial in the security parameter.

2.1 Zero Knowledge Proofs

An argument system for an NP relationship $\mathcal{R}$ is a protocol between a computationally bounded prover $\mathcal{P}$ and a verifier $\mathcal{V}$. At the end of the protocol, $\mathcal{V}$ is convinced by $\mathcal{P}$ that there exists a witness w such that $(x, w) \in \mathcal{R}$ for some input x . Formally, consider the following definition, where we assume $\mathcal{R}$ is known to $\mathcal{P}$ and $\mathcal{V}$.

DEFINITION 1 (ZERO-KNOWLEDGE ARGUMENT OF KNOWLEDGE). Let $\mathcal{R} \subseteq \{0, 1\}^* \times \{0, 1\}^*$ be an NP relation. A zero-knowledge argument of knowledge for $\mathcal{R}$ is a tuple of PPT algorithms (Setup, Prove, Verify) such that:

- $\text{srs} \leftarrow \text{Setup}(1^\kappa)$: on input the security parameter κ , outputs a structured reference string srs .
- $\pi \leftarrow \text{Prove}(\text{srs}, x, w)$: on input public parameters pp , a statement x , and a witness w such that $(x, w) \in \mathcal{R}$, outputs a proof π .
- $b \leftarrow \text{Verify}(\text{srs}, x, \pi)$: on input public parameters srs , a statement x , and a proof π , outputs a bit $b \in \{0, 1\}$ indicating acceptance or rejection.

The scheme satisfies the following properties:

Completeness: For all $(x, w) \in \mathcal{R}$,

$$\Pr \left[\text{Verify}(\text{pp}, x, \pi) = 1 \mid \begin{array}{l} \text{srs} \leftarrow \text{Setup}(1^\kappa) \\ \pi \leftarrow \text{Prove}(\text{pp}, x, w) \end{array} \right] = 1.$$

Knowledge Soundness: For every PPT prover $\mathcal{P}^*$, there exists a PPT extractor $\mathcal{E}$ such that for all x ,

$$\Pr \left[\begin{array}{l} (x, w) \notin \mathcal{R} \\ \wedge \text{Verify}(\text{pp}, x, \pi^*) = 1 \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\kappa) \\ (\pi^*, \text{st}) \leftarrow \mathcal{P}^*(\text{pp}, x) \\ w \leftarrow \mathcal{E}(\text{pp}, x, \pi^*, \text{st}) \end{array} \right] \leq \text{negl}(\kappa),$$

where st denotes $\mathcal{P}^*$'s internal state.

⁴We clarify that our protocols remain designated-verifier due to the use of AHE scheme.

Zero Knowledge: There exists a PPT simulator $\mathcal{S}$ such that for every PPT verifier $\mathcal{V}^*$, $(x, w) \in \mathcal{R}$, and $\text{srs} \leftarrow \text{Setup}(1^\kappa)$,

$$\{\text{View}(\mathcal{V}^*(\text{pp}, x, z; \langle \text{Prove}(\text{srs}, x, w), \mathcal{V}^*(\text{srs}, x, z) \rangle))\} \\ \approx_c \{\mathcal{S}(\text{srs}, x, z)\},$$

where View denotes the view of $\mathcal{V}^*$ during the interaction.

2.2 Polynomial Commitment Schemes

DEFINITION 2 (POLYNOMIAL COMMITMENT SCHEME). A *polynomial commitment scheme (PCS)* is a tuple of algorithms $\text{PCS} = (\text{Setup}, \text{Commit}, \text{Open}, \text{Verify})$ such that:

- $\text{srs} \leftarrow \text{Setup}(\text{pp}, d)$: on input the system parameters pp and a degree bound d , outputs a structured reference string srs .
- $\text{C} \leftarrow \text{Commit}(\text{srs}, f)$: on input srs and a polynomial $f \in \mathbb{F}[X]_{\leq d}$, outputs a commitment C .
- $b \leftarrow \text{Open}(\text{srs}, \text{C}, f)$: input a commitment C and a polynomial f , outputs a bit $b \in \{0, 1\}$ indicating acceptance or rejection of the statement that C opens to f .
- $\pi \leftarrow \text{Eval}(\text{srs}, \text{C}, f, r)$: on input srs , C , f , and a point $r \in \mathbb{F}$, outputs an evaluation proof π that $y = f(r)$.
- $b \leftarrow \text{Verify}(\text{srs}, \text{C}, r, y, \pi)$: on input srs , the commitment C , an evaluation point r , a value $y \in \mathbb{F}$, and an evaluation proof π , outputs a bit $b \in \{0, 1\}$ indicating acceptance or rejection of the statement that $y = f(r)$ w.r.t. C .

We require the following properties.

Binding. For every PPT adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{c} f_0 \neq f_1 \wedge \\ \text{Commit}(\text{srs}, f_0) \\ = \text{Commit}(\text{srs}, f_1) \end{array} \middle| \begin{array}{c} \text{srs} \leftarrow \text{Setup}(\text{pp}, d) \\ (f_0, f_1) \leftarrow \mathcal{A}(\text{srs}) \end{array} \right] \leq \text{negl}(\kappa).$$

Hiding. For every PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr \left[\begin{array}{c} b = b' \end{array} \middle| \begin{array}{c} \text{srs} \leftarrow \text{Setup}(\text{pp}, d) \\ (f_0, f_1, \text{st}) \leftarrow \mathcal{A}_1(\text{srs}) \\ b \leftarrow \{0, 1\} \\ \text{C} \leftarrow \text{Commit}(\text{srs}, f_b) \\ b' \leftarrow \mathcal{A}_2(\text{srs}, \text{C}, \text{st}) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\kappa).$$

Completeness. For all $f \in \mathbb{F}[X]_{\leq d}$, all $r \in \mathbb{F}$, and all $y = f(r)$,

$$\Pr \left[\begin{array}{c} \text{Verify}(\text{srs}, \text{C}, r, y, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{srs} \leftarrow \text{Setup}(\text{pp}, d) \\ \text{C} \leftarrow \text{Commit}(\text{srs}, f) \\ \pi \leftarrow \text{Eval}(\text{srs}, \text{C}, f, r) \end{array} \right] = 1.$$

Extractability. For every PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that for all degree bounds d ,

$$\Pr \left[\begin{array}{c} \text{Verify}(\text{srs}, \text{C}, r, y, \pi) = 1 \\ \wedge f(r) \neq y \end{array} \middle| \begin{array}{c} (\text{pp}, d) \\ \text{srs} \leftarrow \text{Setup}(\text{pp}, d) \\ (\text{C}, r, y, \pi) \leftarrow \mathcal{A}(\text{srs}) \\ f \leftarrow \mathcal{E}(\text{srs}, \text{C}) \end{array} \right] \leq \text{negl}(\lambda).$$

2.3 Additively Homomorphic Encryption

DEFINITION 3 (ADDITIVELY HOMOMORPHIC ENCRYPTION [42]). An *additive homomorphic encryption (AHE)* scheme is a tuple of algorithms $\text{AHE} = (\text{Setup}, \text{Gen}, \text{Enc}, \text{Dec})$ such that:

- $\text{pp} \leftarrow \text{Setup}(1^\kappa)$: on input the security parameter κ , outputs a public parameter pp .
- $\text{sk} \leftarrow \text{Gen}(\text{pp})$: on input the public parameter pp outputs a secret key sk .
- $c \leftarrow \text{Enc}(\text{sk}, m)$: on input the secret key sk and a message $m \in \mathbb{F}$, outputs a ciphertext c .
- $m \leftarrow \text{Dec}(\text{sk}, c)$: on input the secret key sk and a ciphertext c , outputs a message m .
- Additively homomorphic operations: there exists an efficiently computable operation on ciphertexts such that decryption of the ciphertext-level sum equals the sum of the corresponding plaintexts. That is, for any ciphertexts $c_1 = \text{Enc}(\text{sk}, m_1)$, $c_2 = \text{Enc}(\text{sk}, m_2)$, $\text{Dec}(\text{sk}, c_1 + c_2) = m_1 + m_2$.

We also write $\langle m \rangle = \text{Enc}(\text{pk}, m)$ for the encryption of a message m . The additive homomorphism extends to linear combinations: for any scalars $a, b \in \mathbb{F}$ known by $\mathcal{P}$ and any messages m_1, m_2 in the message space, decryption of the ciphertext w.r.t. $a \cdot \langle m_1 \rangle + b \cdot \langle m_2 \rangle$ yields $a \cdot m_1 + b \cdot m_2$.

2.4 Information-Theoretic MAC

We start with vector oblivious linear evaluation (VOLE), a.k.a., information-theoretic message authentication codes (IT-MACs) [8, 35] (we'll use the two names interchangeably). Specifically, consider two parties $\mathcal{P}$ and $\mathcal{V}$. Given a secret message $x \in \mathbb{F}$, a MAC m of x satisfies $m = k + \Delta \cdot x$ where $\Delta, k \in \mathbb{F}$ are uniformly random *global key* and *local key* held by $\mathcal{V}$, and the message x as well as the MAC tag m are held by $\mathcal{P}$. We denote such an authenticated value by $[x]$.

Generation of MACs. To generate a MAC of x , $\mathcal{P}, \mathcal{V}$ first prepare a random authenticated value $[r]$ (i.e., a VOLE correlation) where $m^* = k^* + \Delta \cdot r$. Then $\mathcal{P}$ sends a masked value $s = x - r$ to $\mathcal{V}$, and $\mathcal{P}, \mathcal{V}$ locally set $m = m^*$ and $k = k^* - \Delta \cdot s$, respectively. Clearly, $k = k^* - \Delta(x - r) = m^* - \Delta \cdot x$, and thus m is a valid MAC tag of x .

Linearity. A fundamental property of IT-MACs is *linearity* (or additive homomorphism), which allows parties to locally compute IT-MACs for linear combinations of authenticated values. Specifically, given $[x], [y]$ with MAC tags $m_x = k_x + \Delta \cdot x$, $m_y = k_y + \Delta \cdot y$ and public scalars $a, b \in \mathbb{F}$, the parties can obtain a valid MAC for $z = a \cdot x + b \cdot y$ without interaction: $\mathcal{P}$ computes $m_z = a \cdot m_x + b \cdot m_y$, while $\mathcal{V}$ computes $k_z = a \cdot k_x + b \cdot k_y$. This property is crucial for VOLE-based protocols as it enables the parties to locally derive some IT-MACs and compute addition gates freely.

Zerocheck on MACs. In VOLE-based ZKPs, we'll need to check if an authenticated value $[x]$ is zero. Note that if $x = 0$, it must hold that $m = k$. Thus, $\mathcal{P}$ can directly send m to $\mathcal{V}$, who then checks if it is consistent with k . For checking multiple values $[x_1], \dots, [x_n]$, $\mathcal{P}$ can instead send a hash of $m_1, \dots, m_n$ to $\mathcal{V}$ who then compares it with her local hash of $k_1, \dots, k_n$ to save communication.

2.5 Security Model and Functionality

We analyze our protocols in the random oracle model with ideal access to the VOLE functionality $\mathcal{F}_{\text{VOLE}}$. It provides the prover and verifier with correlated VOLE randomness during preprocessing. We give a formal definition of $\mathcal{F}_{\text{VOLE}}$ in Figure 1.

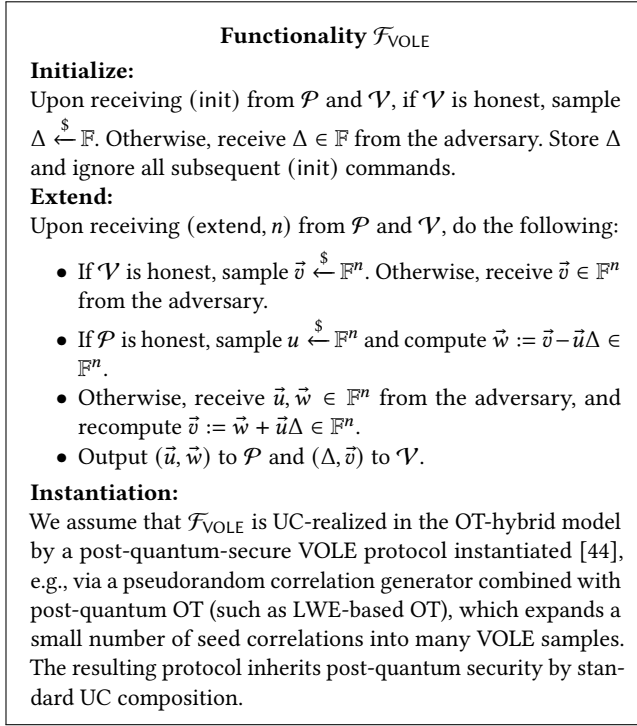


Figure 1: Functionality for VOLE correlations.

2.6 Univariate Sumcheck

We use univariate sumcheck techniques from existing works [7, 16] for constructing Wasp^C. We review these techniques in Appendix A.

3 A New PCS From VOLE

In this section, we present a new PCS based on VOLE, which enhances IT-PACs with the support of evaluation proofs.

3.1 Review: IT-PACs from Antman

IT-PACs [42] generalize IT-MACs by allowing the prover to authenticate a secret polynomial $f \in \mathbb{F}_{\leq d}[X]$ rather than a value x . To this end, the verifier $\mathcal{V}$ samples a secret randomness $\Lambda \leftarrow \mathbb{F}$ (called “polynomial key” in Antman [42]), and then $\mathcal{P}, \mathcal{V}$ generate a MAC tag M of the evaluation $f(\Lambda)$ as the authentication of f , i.e., $M = K + \Delta \cdot f(\Lambda)$. Note that since Λ is secret from $\mathcal{P}$ and f is secret from $\mathcal{V}$, the evaluation $f(\Lambda)$ is kept secret from both parties.

In Antman, IT-PAC is constructed with the cryptographic tool AHE. We briefly review the procedure. The goal is to let the parties obtain an authenticated evaluation $[f(\Lambda)]$ where f, Λ are held by $\mathcal{P}, \mathcal{V}$ respectively, and $f(\Lambda)$ is unknown to any party. Assume the degree of f is d . After $\mathcal{V}$ samples Λ , we let $\mathcal{V}$ encrypt $\{\Lambda^i\}_{i=1}^{d-1}$ using an AHE scheme, and then send all the ciphertexts, denoted by $\{\langle \Lambda^i \rangle\}_{i=1}^{d-1}$, to $\mathcal{P}$. The prover $\mathcal{P}$ then computes and returns an encryption of a masked value $s = f(\Lambda) - r$ based on these ciphertexts, i.e., $\langle s \rangle = f_0 + \sum_{i=1}^{d-1} f_i \cdot \langle \Lambda^i \rangle - r$ where $\{f_i\}_{i=0}^{d-1}$ are the coefficients of f , and r is an authenticated randomness w.r.t. a prepared MAC $m^* = k^* + \Delta \cdot r$ (recall Section 2.4). $\mathcal{V}$ then decrypts the ciphertext to attain s , and the parties can locally obtain $[f(\Lambda)]$ using the prepared MAC $[r]$ by setting $M = m^*$ and $K = k^* - \Delta \cdot s$.

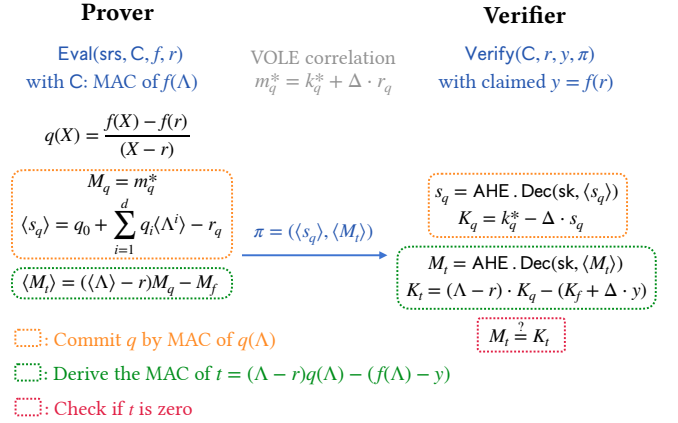


Figure 2: The procedure of our evaluation proof.

3.2 Our New PCS

Intuitively, we can view a PAC $[f(\Lambda)]$ as a commitment to the polynomial f , and it satisfies the hiding and binding properties of PCS.⁵ However, compared to the PCS adopted by modern ZKP systems, IT-PACs cannot support opening the polynomial at arbitrary verifier-chosen points.⁶ Thus, we aim to enhance IT-PACs to a fully functional PCS with the support of generic evaluation openings.

Our Techniques. We show the procedure of our evaluation proof in Figure 2, and elaborate on it below. Our key observation is that given the authenticated secret evaluation $[f(\Lambda)]$, we can rely on the standard polynomial divisibility property adopted by existing PCS (like KZG10-style ones [28, 36]) to prove the correctness of the evaluation $f(r)$. To be more specific, for any polynomial $f \in \mathbb{F}_{\leq d}[X]$ and any $r \in \mathbb{F}$, the polynomial $X - r$ must divide $f(X) - f(r)$, since $y = f(r)$ is a valid evaluation of f at r iff $X - r$ is a root of the polynomial $f(X) - y$. Hence, there must exist a witness polynomial $q(X)$ of degree $\deg(f) - 1$ s.t. $f(X) - y = (X - r)q(X)$. Given the Schwartz-Zippel lemma [37, 46], it suffices to check the polynomial equation at any verifier-chosen random point, e.g., Λ .

More concretely, given f , we let the prover $\mathcal{P}$ generate a PAC $[f(\Lambda)]$ as its commitment. Then, for any claimed evaluation $y = f(r)$, we ask $\mathcal{P}$ to compute a witness polynomial $q(X)$ s.t. $q(X) = (f(X) - y)/(X - r)$, and commit to $q(X)$ to obtain a PAC $[q(\Lambda)]$. Then $\mathcal{V}$ can verify if $f(\Lambda) - y = (\Lambda - r)q(\Lambda)$ using VOLE techniques.

However, one problem arises during this verification procedure. Specifically, the above check is reduced to a zerocheck on the value

$$[t] = (\Lambda - r)[q(\Lambda)] - ([f(\Lambda)] - y).$$

Let M_f, M_q and K_f, K_q be the MAC tags and local keys of $f(\Lambda), q(\Lambda)$ respectively. Since MACs are additively homomorphic, it seems that the two parties can locally derive the MAC of t by

- $\mathcal{P}$ computing $M_t = (\Lambda - r)M_q - M_f$, and
- $\mathcal{V}$ computing $K_t = (\Lambda - r)K_q - (K_f + \Delta y)$,

⁵Informally, hiding holds since $f(\Lambda)$ is masked by a random value r , and thus f remains secret from $\mathcal{V}$. Binding holds because the polynomial key Λ (as well as Δ, K) is only known to $\mathcal{V}$, and it is hard for $\mathcal{P}$ to forge an incorrect PAC to make the relation hold (given that $\mathbb{F}$ is sufficiently large).

⁶In fact, it only supports opening f at a single verifier-chosen point Λ by directly revealing Λ to the prover and then checking the authenticity of the MAC tag, which is the main cause of Antman being interactive.

and then perform a zerocheck procedure on $[t]$, in which $\mathcal{P}$ will send M_t to $\mathcal{V}$ and $\mathcal{V}$ directly checks if $t = 0$ by comparing if $M_t = K_t$. However, $\mathcal{P}$ is unable to compute M_t because Λ is held only by $\mathcal{V}$, and we don't want Λ to be revealed to $\mathcal{P}$.

To resolve the problem, instead of letting $\mathcal{P}$ directly send M_t to $\mathcal{V}$, we can alternatively rely on the AHE scheme again to let $\mathcal{P}$ compute an encryption of M_t using the ciphertext $\langle \Lambda \rangle$, i.e.,

$$\langle M_t \rangle = (\langle \Lambda \rangle - r)M_q - M_f,$$

and send it to $\mathcal{V}$ as well. After this, $\mathcal{V}$ is able to decrypt M_t , and then complete the zerocheck on $[t]$.

Our evaluation proof has $O(1)$ size (i.e., two AHE ciphertexts) and verifier time (i.e., two AHE decryption operations and minor field operations). It also shows similar merits to the KZG10 PCS [28], e.g., being friendly to batch evaluation proofs for multiple polynomials [20] by using random linear combinations. We provide formal descriptions and complete security proofs for our PCS and its batching scheme in Appendix C.

Application of Our PCS. The above PCS can be viewed as a standalone component for constructing ZKP schemes, following the modern "PIOP + PCS" paradigm. In this work, we use this PCS to construct Wasp^S and Wasp^G.

Constructing Wasp^S. We optimize the SIMD ZKP of Antman with our PCS to eliminate its requirement for interaction, and improve its concrete efficiencies. We describe this optimization in Section 4.

Constructing Wasp^G. Another direct application is using our PCS to replace the original KZG10 PCS [28] in the Plonk zkSNARK [21], and obtain a (designated-verifier) zkSNARK with constant proof size and verification time. Since the replacement is straightforward, we provide a detailed construction in Appendix E.

4 An Optimized ZKP for SIMD Circuits

We now elaborate on Wasp^S, a non-interactive ZKP for SIMD circuits with higher concrete efficiencies than Antman.

4.1 Review: SIMD ZKP of Antman

We first briefly review the SIMD ZKP of Antman from the underlying coordinate-wise relation. For a circuit consisting of B instances of a subcircuit C , Antman encodes the B evaluations on every wire in C into a polynomial, then commits it with the IT-PAC scheme and checks the circuit gate-by-gate using IT-PACs. Specifically, we fix one multiplication-gate position in subcircuit C . The left inputs, right inputs, and outputs of this gate among the B executions form three vectors $\vec{l}, \vec{r}, \vec{o} \in \mathbb{F}^B$, and correctness means $\vec{o}[i] = \vec{l}[i] \cdot \vec{r}[i]$ for every $i \in [B]$. $\mathcal{P}$ encodes them into three degree- $(B-1)$ polynomials $f, g, h \in \mathbb{F}_{\leq B-1}[X]$ s.t. $f(i) = \vec{l}[i]$, $g(i) = \vec{r}[i]$, and $h(i) = \vec{o}[i]$ for $i \in [B]$, and then commits them using IT-PACs. Since the formal product polynomial $f \cdot g$ has degree $2B-2$, it is not equal to the degree- $(B-1)$ interpolated polynomial h , even when all B multiplications are correct. Antman therefore let the prover compute and commit to $\tilde{h}(X) = f(X) \cdot g(X)$ using an IT-PAC and then reduces the coordinate-wise multiplication relation to the following two conditions:

- (1) $\tilde{h}(X) = f(X) \cdot g(X)$, and
- (2) $\tilde{h}(i) = h(i), \forall i \in [B]$.

To check the first condition, it suffices for $\mathcal{V}$ to verify the polynomial equation at a random point given the Schwartz-Zippel lemma [37, 46]. As the IT-PAC scheme of Antman cannot support evaluating the committed polynomials at an arbitrary random point, the only available evaluation point is precisely the verifier's secret IT-PAC key Λ . Thus, $\mathcal{V}$ is asked to reveal Λ to $\mathcal{P}$, who then can evaluate the polynomials at Λ and acquire the value $f(\Lambda)$, and further prove $[\tilde{h}(\Lambda)] = [f(\Lambda)] \cdot [g(\Lambda)]$ using VOLE-based multiplication check techniques like Quicksilver [33, 43]. Revealing Λ is sound because the polynomials are already bound, but Λ must be discarded afterwards.

To check the second condition, $\mathcal{P}$ directly opens the two polynomials to $\mathcal{V}$ (with masks), who then locally checks the relation. Additionally, $\mathcal{V}$ has to verify if the received polynomials are correct, i.e., consistent with their IT-PACs. Since Λ has been revealed, the check can be naturally reduced to zerochecks (i.e., checking if both $[t] = [h(\Lambda)] - h(\Lambda)$ and $[\tilde{t}] = [\tilde{h}(\Lambda)] - \tilde{h}(\Lambda)$ are a MAC of zero).

4.2 Our Optimization

Note that the interaction between $\mathcal{P}, \mathcal{V}$ in Antman cannot be removed via Fiat-Shamir transformation [19], Fiat-Shamir can replace a fresh public verifier-chosen challenge with a transcript-derived one, whereas Λ is fixed and used while generating the IT-PACs (i.e., in the committing phase), and thus cannot be transformed by Fiat-Shamir. We avoid revealing Λ by replacing Antman's checks with a multiplication check PIOP and compiling it with our PCS to enable testing the polynomial identities at a fresh public challenge point. This challenge can then be generated by Fiat-Shamir, and thus making the online proof phase non-interactive.

As aforementioned, we need to check if $h(i) = f(i) \cdot g(i)$ holds for all $i \in [B]$ where f, g, h encode the wire values of a multiplication gate's left, right input and output in the B subcircuits, respectively. To finish this check, we can alternatively use the existing PIOP for multiplication relations. W.l.o.g., we assume $\mathbb{H}$ is a smooth multiplicative subgroup of $\mathbb{F}$ with order B and generator ω . We now encode the wire values over $\mathbb{H}$ to obtain f, g, h . Then $h(\omega^i) = f(\omega^i) \cdot g(\omega^i)$ holds for all $i \in [B]$, which implies that $\{\omega^i\}_{i=0}^{B-1}$ are the B roots of the polynomial $f(X) \cdot g(X) - h(X)$. Let $Z_{\mathbb{H}}(X) = \prod_{i=0}^{B-1} (X - \omega^i) = X^B - 1$ be the vanishing polynomial over $\mathbb{H}$. Then there must exist a quotient polynomial $Q(X)$ of degree $B-2$ s.t.

$$f(X) \cdot g(X) - h(X) = Q(X)Z_{\mathbb{H}}(X).$$

Thus, it suffices for $\mathcal{P}$ to compute and commit Q , and then let $\mathcal{V}$ randomly sample a challenge point r to test if the polynomial equation holds at r .

The above check procedure can be batched using random linear combinations when processing C multiplication gates, and after batching, only one combined quotient polynomial needs to be committed. The total proof size is $O(C)$ ciphertexts (incurred by committing C groups of f, g, h) plus $O(C)$ field elements (i.e., the evaluations of the committed polynomials at the point r). The verifier cost is $O(C)$ AHE decryptions and $O(C)$ field operations. We thus can achieve succinctness by setting $C = o(N)$. Notice that in the above protocol, the secret polynomial key Λ will never be revealed, and we can naturally turn the protocol into non-interactive with the Fiat-Shamir transformation. We give a formal description

PROTOCOL 3.1. $\Pi_{\text{ZK}}^{\text{SIMD}}$ (Our Wasp^S Protocol for SIMD Circuits)

Let C be a circuit over $\mathbb{F}$ with $C = |C|$ multiplication gates and m input gates. $\mathcal{P}$ holds B satisfying assignments (witnesses) to C , denoted by $\vec{w}_1, \dots, \vec{w}_B \in \mathbb{F}^m$ s.t. $C(\vec{w}_i) = 0$ for $i \in [B]$. W.l.o.g., let $\mathbb{H}$ be a multiplicative subgroup of $\mathbb{F}$ with order B and generator ω . Let $Z_{\mathbb{H}}(X) = X^B - 1$ be the vanishing polynomial over $\mathbb{H}$. Let PCS denote the batched PCS $\Pi_{\text{PCS-Batch}}$ described in Protocol C.2.

- **Setup:** $\mathcal{P}, \mathcal{V}$ run $\text{PCS.Setup}(1^\kappa)$ with supported degree $d = B - 1$.
- **Prove:**
 - (1) For each $j \in [m]$, $\mathcal{P}$ encodes the j -th group of B input values $\{\vec{w}_i[j]\}_{i=0}^{B-1}$ into a degree- $(B-1)$ polynomial u_j s.t. $u_j(\omega_i) = \vec{w}_i[j]$. $\mathcal{P}$ runs $\text{PCS.Commit}(\text{srs}, \{u_j\}_{j=0}^{m-1})$ to generate commitments to $\{u_j\}_{j=0}^{m-1}$.
 - (2) Following a predetermined topological order of circuit C ,
 - (a) For each group of addition gates with input polynomial commitments C_f, C_g , $\mathcal{P}, \mathcal{V}$ locally derive an output polynomial commitment $C_h = C_f + C_g$.
 - (b) For j -th group of multiplication gates where $j \in [C]$ with input polynomial commitments C_{f_j}, C_{g_j} , let h_j be the output polynomial encoding the B output values.
 - (i) $\mathcal{P}, \mathcal{V}$ run $\text{PCS.Commit}(\text{srs}, \{h_j\}_{j=0}^{C-1})$ to generate commitments $\{C_{h_j}\}_{j=0}^{C-1}$.
 - (ii) $\mathcal{V}$ samples and sends $\eta \xleftarrow{\$} \mathbb{F}$ to $\mathcal{P}$.
 - (iii) $\mathcal{P}$ computes a quotient polynomial $Q \in \mathbb{F}_{\leq B-2}[X]$ satisfying $\sum_{j=0}^{C-1} \eta^j (f_j(X)g_j(X) - h_j(X)) = Q(X)Z_{\mathbb{H}}(X)$, and $\mathcal{P}, \mathcal{V}$ run $\text{PCS.Commit}(\text{srs}, Q)$ to obtain C_Q .
 - (iv) $\mathcal{V}$ samples and sends $r \xleftarrow{\$} \mathbb{F} \setminus \mathbb{H}$.
 - (v) Let $F = \{\{f_j, g_j, h_j\}_{j=0}^{C-1}, Q\}$, and $Y = \{\{f_j(r), g_j(r), h_j(r)\}_{j=0}^{C-1}, Q(r)\}$. $\mathcal{P}$ runs $\pi = \text{PCS.BatchEval}(\text{srs}, F, r)$, and sends (Y, π) to $\mathcal{V}$.
 - **Verify:** Let $C = \{\{C_{f_j}, C_{g_j}, C_{h_j}\}_{j=0}^{C-1}, C_Q\}$. $\mathcal{V}$ runs $b_\pi = \text{PCS.BatchVerify}(\text{srs}, C, r, Y, \pi)$, and checks if $\sum_{j=0}^{C-1} \eta^j (f_j(r) \cdot g_j(r) - h_j(r)) = Q(r)Z_{\mathbb{H}}(r)$ holds. If $b_\pi = 1$ and the check pass, $\mathcal{V}$ outputs $b = 1$, otherwise $b = 0$.

of Wasp^S in Protocol 3.1. Its complete security proof is provided in Appendix D.

5 A Succinct SIMD-to-General ZK Compiler

In this section, we present an optimized compiler for compiling a SIMD ZKP to a general one, by designing a new wire consistency check protocol with a succinct verifier.

5.1 Review: The Compiler of Antman++

The high-level idea of the SIMD-to-general ZK compiler from Antman++ [12] follows the one from Antman [42]: given an arbitrary circuit, it is first transformed into an equivalent one (computing the same functionality) that consists of SIMD subcircuits, and then proved using a SIMD ZKP; additionally, it is applied to a wire consistency check to guarantee the satisfaction of wire connections among the subcircuits.

Circuit Transformation from General to SIMD. We first briefly review how to transform a general circuit into an equivalent one composed of SIMD subcircuits by padding dummy gates, following Antman++ [12, 42]. Without loss of generality, we assume the general circuit has arbitrary numbers of inputs and arithmetic gates, and one output gate. Let C_M and C_A denote the numbers of multiplication and addition gates in this circuit, respectively. Let B be the parameter describing the number of transformed SIMD subcircuits, and $C = \lceil C_M/B \rceil$ the number of multiplication gates of each resulting SIMD subcircuit. The high-level idea is to reorganize the circuit to make it divisible by B parallel subcircuits with dummy gates and wires. Specifically, we append $\lceil C_M/B \rceil \cdot B - C_M$ dummy multiplication gates and $\lceil C_A/B \rceil \cdot B - C_A$ dummy addition gates to the original circuit, where all newly introduced auxiliary wires

are assigned the value zero, so that the numbers of multiplication and addition are both divisible by B . After this step, if the number of output gates is less than B , we append dummy input gates with zero wire values and also define them as output gates, to ensure the number of output gates is (at least) B ; if the number of input gates are not a multiple of B , we repeat to pad dummy input gates (that are defined to be output gates as well) to make the total number of input gates is divisible by B . Consequently, the transformation preserves the computation on all original wires while increasing the circuit size by only $O(B)$.

After padding, the resulting circuit can be viewed as B parallel executions of the same subcircuit. We then partition the padded circuit in a predetermined topological order and collect every B gates of the same arithmetic type into one group. Specifically, we divide and collect every B input, multiplication, and addition gates into a group. For output gates, we collect the original actual output gate along with $B - 1$ dummy output gates into a group. This yields a total of C groups of multiplication gates where each group has size B , which can be proved using the underlying SIMD ZKP.

While this establishes the correct execution of the SIMD subcircuits, it does not by itself enforce the original circuit topology. In particular, the same circuit wire may appear at multiple gate positions and hence be collected in different groups. Therefore, a SIMD-to-general compiler must additionally enforce wire consistency among these subcircuits.

Wire Consistency Check in Antman++. We sketch the wire consistency check protocol of Antman++ below. Assume a general circuit of Bm wire values, and let $\vec{u} \in \mathbb{F}^{Bm}$ contain the values for all global circuit wires. Let $\vec{l}, \vec{r}, \vec{o} \in \mathbb{F}^{BC}$ be the values on the left, right input wires and output wires of the BC multiplication gates,

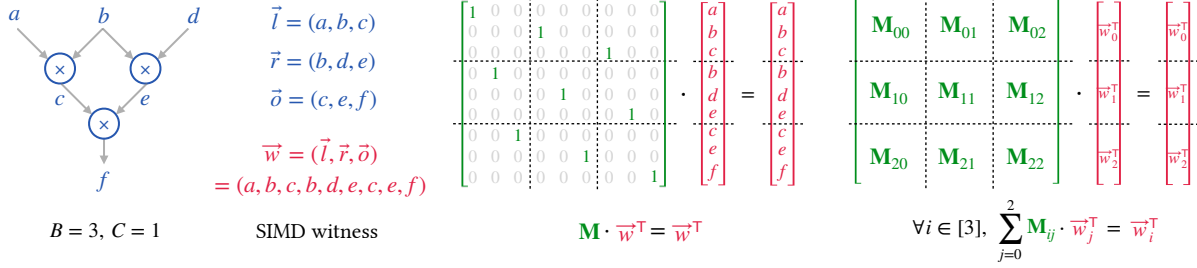


Figure 3: An illustrative example of the high-level idea of our wire consistency check.

respectively. If all the wire values are consistent, then there must be three public sparse matrices $\mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{F}^{BC \times Bm}$ s.t. the following matrix-vector product equalities holds: $\vec{l} = \mathbf{L} \cdot \vec{u}, \vec{r} = \mathbf{R} \cdot \vec{u}$ and $\vec{o} = \mathbf{O} \cdot \vec{u}$, where the three public matrices $\mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{F}^{BC \times Bm}$ encode the circuit topology: each row selects from $\vec{u}$ the global wire connected to the corresponding gate position. To avoid checking these matrix-vector product equalities directly, AntMan++ samples a random vector $\vec{\rho} \in \mathbb{F}^{BC}$ and reduces the above matrix-vector relations to inner-product relations, i.e., $\vec{\rho}^T \cdot \vec{l} = (\vec{\rho}^T \mathbf{L}) \cdot \vec{u}$. Let $\vec{v}^T = \vec{\rho}^T \mathbf{L}$ be a public vector, the relation turns to a length- BC inner-product relation $\vec{\rho}^T \cdot \vec{l} = \vec{v}^T \cdot \vec{u}$ (analogously to $\vec{r}$ and $\vec{o}$). Antman++'s key insight is to further divide the length- BC inner-product computation into B instances of a length- C inner-product circuit following a summation on the B inner-product results. The SIMD inner-product computation can again be handled by the underlying SIMD ZKP, and the final summation is proved by $\mathcal{P}$ directly opening some related polynomial to $\mathcal{V}$, who then locally performs the final check.

5.2 Our Optimized Compiler

We observe that in Antman++, the verifier has to compute three public vectors $\vec{\rho}^T \mathbf{L}, \vec{\rho}^T \mathbf{R}$ and $\vec{\rho}^T \mathbf{O}$, which involves $O(N)$ computational cost. Besides, both the global witness $\vec{u}$ and the wire values $\vec{l}, \vec{r}, \vec{o}$ have to be committed, increasing redundant prover cost for committing witness. Our goal is to achieve succinctness without incurring high prover efficiency loss.

We give an illustrative example in Figure 3 to show our high-level idea. Consider the illustrative circuit, which consists of three multiplication gates. We divide this circuit into three SIMD subcircuits each computing one multiplication gate, and thus $B = 3$ and $C = 1$. The three SIMD subcircuits are proved by the underlying SIMD ZKP, where we assume the witnesses are collected as in Antman (and also Wasp⁵, recall Section 4.1), i.e., we obtain three vectors $\vec{l}, \vec{r}, \vec{o} \in \mathbb{F}^3$ which are the wire values of the left input, right input and output of the multiplication gate in the three subcircuits. To avoid redundant witness committing, we organize these witnesses in a way that aligns with the underlying SIMD ZKP. More specifically, let $\vec{w} = (\vec{l}, \vec{r}, \vec{o}) \in \mathbb{F}^9$; our key observation is that the wire values satisfy the circuit structure iff there exists a public sparse matrix $\mathbf{M} \in \mathbb{F}^{9 \times 9}$ (where there is only one non-zero value 1 in each row or column) s.t. $\mathbf{M} \vec{w}^T = \vec{w}^T$. Then, to keep alignment with the underlying SIMD ZKP, we split the matrix $\mathbf{M}$ into nine blocks and divide the vector $\vec{w}$ into three smaller subvectors (as shown in Figure 3). Then the relation is reduced to $\sum_{j=0}^2 \mathbf{M}_{ij} \vec{w}_j^T = \vec{w}_i^T, \forall i \in [3]$.

In this way, we can avoid introducing other witness vectors (such as $\vec{u}$ as in Antman++), and save redundant prover cost.

Existing works [15, 16] have explored preprocessing techniques for proving a sparse matrix-vector product relation to achieve succinctness. However, as we work in the SIMD setting, we have to prove a batch of (specifically, $3C$) sparse matrix-vector product summation (of length B), and a trivial application of existing techniques would incur a linear verifier cost. We thus extend and explore new preprocessing techniques for the SIMD setting to achieve succinctness. We elaborate on our techniques below.

(1) Reorganize as SIMD Matrix-Vector Multiplication. We now consider the general case. For an arbitrary circuit, we pad it into a SIMD one consisting of B instances of a subcircuit C . As aforementioned, for each i -th multiplication gate in C where $i \in [C]$, we collect all the values on the left, right, and output wires of this gate in the B subcircuits as $\vec{l}_i, \vec{r}_i, \vec{o}_i \in \mathbb{F}^B$. Let $\vec{w} = (\{\vec{l}_i, \vec{r}_i, \vec{o}_i\}_{i \in [C]}) \in \mathbb{F}^{3BC}$. Then wire consistency holds iff there exists a public matrix $\mathbf{M} \in \mathbb{F}^{3BC \times 3BC}$ s.t. $\mathbf{M} \vec{w}^T = \vec{w}^T$. We reorganize the witness into $3C$ length- B subvectors $\{\vec{w}_i^T\}_{i \in [3C]}$, and also partition the matrix into $3C \times 3C$ smaller ones $\{\mathbf{M}_{ij}\}_{i,j \in [3C]}$, each of size $B \times B$. Then the consistency condition is turned into

$$\sum_{j=0}^{3C-1} \mathbf{M}_{ij} \vec{w}_j^T = \vec{w}_i^T, \forall i \in [3C]. \quad (1)$$

(2) Encode $\mathbf{M}_{ij}$ With Sparse Representation. In this step, we follow previous techniques [15, 16] to encode each submatrix $\mathbf{M}_{ij}$ with a sparse representation. Note that each submatrix $\mathbf{M}_{ij}$ inherits the sparse property of the matrix $\mathbf{M}$, and contains at most B non-zero entries. Below for simplicity of exposition, we pad both dimensions B and $3C$ to a common size D and index them using a common domain $\mathbb{H}$, which is a smooth multiplicative subgroup of $\mathbb{F}$ with order D and generator ω . We'll also use another multiplicative subgroup $\mathbb{K}$ with order $B \leq D$ to encode the non-zero entries (as we'll show later). We first clarify how we index the entries in $\mathbf{M}_{ij}$. Since $\mathbf{M}_{ij}$ is of size $D \times D$, we can use elements in $\mathbb{H}$ to index its entries, i.e., for $a, b \in \mathbb{H}$ where $a = \omega^{i_a}, b = \omega^{i_b}$, $\mathbf{M}_{ij}[a, b]$ indicates the entry located at the i_a -th row and i_b -th column. We now introduce a bivariate polynomial

$$u_{\mathbb{H}}(X, Y) = \frac{Z_{\mathbb{H}}(X) - Z_{\mathbb{H}}(Y)}{X - Y},$$

which is the formal derivative of the vanishing polynomial $Z_{\mathbb{H}} = X^D - 1$. It has a key property that it vanishes on the square $\mathbb{H} \times \mathbb{H}$ except for on the diagonal, i.e., it only evaluates to non-zero values $u_{\mathbb{H}}(a, a)$ when $X = Y = a \in \mathbb{H}$. Given this, we can encode

the non-zero entries of each submatrix $\mathbf{M}_{ij}$ with a sparse representation. Concretely, we introduce three univariate polynomials $\text{row}_{ij}, \text{col}_{ij}, \text{val}_{ij} \in \mathbb{F}_{\leq B}[X]$ which are the (unique) low-degree extensions of the three functions $\text{row}_{ij}, \text{col}_{ij}, \text{val}_{ij} : \mathbb{K} \rightarrow \mathbb{F}$ respectively representing the row index, column index of each non-zero entry, and the value of this entry (i.e., 1 in our case) divided by $u_{\mathbb{H}}(\text{row}_{ij}(k), \text{row}_{ij}(k)) \cdot u_{\mathbb{H}}(\text{col}_{ij}(k), \text{col}_{ij}(k))$. Thus, we can encode $\mathbf{M}_{ij}$ into a bivariate polynomial $\hat{M}_{ij}(X, Y)$

$$\hat{M}_{ij}(X, Y) = \sum_{k \in \mathbb{K}} u_{\mathbb{H}}(X, \text{row}_{ij}(k)) u_{\mathbb{H}}(Y, \text{col}_{ij}(k)) \hat{\text{val}}_{ij}(k). \quad (2)$$

By construction, $u_{\mathbb{H}}(X, \text{row}_{ij}(k))$ only evaluates to a non-zero value $u_{\mathbb{H}}(\text{row}_{ij}(k), \text{row}_{ij}(k))$ when $X = \text{row}_{ij}(k)$ (analogously to $\hat{\text{col}}_{ij}$), and thus $\hat{M}_{ij}(X, Y) = \mathbf{M}_{ij}[X, Y] = 1$ for $(X, Y) \in \mathbb{H} \times \mathbb{H}$.

(3) Reduce to Sumcheck Problem. Given the sparse representation of $\mathbf{M}_{ij}$, we turn Equation (1) into

$$\sum_{j=0}^{D-1} \sum_{Y \in \mathbb{H}} \hat{M}_{i,j}(X, Y) \hat{w}_j(Y) = \hat{w}_i(X), \quad \forall X \in \mathbb{H}, i \in [D],$$

where the polynomial $\hat{w}_j(Y)$, $\hat{w}_i(X)$ are the low-degree extensions of the witness $\tilde{w}_j^T$ and $\tilde{w}_i^T$, respectively. We next follow existing techniques [15, 16] to reduce the above check to a sumcheck problem using the idea of random linear combination, by introducing a polynomial $r(X, Y)$ with degree less than D in X and Y such that $r(X, Y)|_{X \in \mathbb{H}}$ form a linearly independent set (the polynomial $u_{\mathbb{H}}$ satisfies the property, and we can simply set $r = u_{\mathbb{H}}$). Then we reduce it to the following sumcheck problem

$$\sum_{j=0}^{D-1} \sum_{Y, X \in \mathbb{H}} r(\alpha_1, Y) \hat{M}_{i,j}(Y, X) \hat{w}_j(X) = \sum_{X \in \mathbb{H}} r(\alpha_1, X) \hat{w}_i(X), \quad \forall i \in [D], \quad (3)$$

where α_1 is a random challenge picked by the verifier.

(4) Batch Prove D Sumcheck. As we work in the SIMD setting, it yields a total of D sumcheck relations. We next bundle these D sumchecks together. To this end, for each $j \in [D]$, we consider a matrix $\mathbf{M}_j$ that consists of all submatrices $\{\mathbf{M}_{ij}\}_{i \in [D]}$. Since there is at most one non-zero entry in each row or column, the matrix $\mathbf{M}_j$ contains at most D non-zero entries. For presentation clarity, we reshape $\mathbf{M}_j$ into a three-dimensional tensor: for each $a, b, c \in \mathbb{H}$ where $a = \omega^{ia}$, $b = \omega^{ib}$, and $c = \omega^{ic}$, $\mathbf{M}_j[a, b, c]$ indicates the entry located at the i_a -th row, i_b -th column of the i_c -th submatrix $\mathbf{M}_{i_c j}$. We thus introduce a new mapping $\hat{\text{mat}}_j$ to indicate the new dimension c . In more detail, for each $k \in \mathbb{K}$ w.r.t. some non-zero entry, $\hat{\text{mat}}_j(k)$ characterizes the new dimension $c = \omega^{ic}$ s.t. the non-zero entry lies in the submatrix $\mathbf{M}_{i_c j}$. We keep using $\text{row}_j(k)$, $\text{col}_j(k)$ to indicate the row, column index of this entry in $\mathbf{M}_{i_c j}$ as defined before, and $\text{val}_j(k)$ is updated by additionally dividing $u_{\mathbb{H}}(\text{mat}_j(k), \text{mat}_j(k))$. We thus encode $\mathbf{M}_j$ with the following trivariate polynomial

$$\hat{M}_j(X, Y, Z) = \sum_{k \in \mathbb{K}} u_{\mathbb{H}}(X, \text{row}_j(k)) u_{\mathbb{H}}(Y, \text{col}_j(k)) u_{\mathbb{H}}(Z, \hat{\text{mat}}_j(k)) \hat{\text{val}}_j(k). \quad (4)$$

Given this representation, we can use a bivariate polynomial $\eta(X, Z)$ such that $\eta(X, Z)|_{Z \in \mathbb{H}}$ from a linearly independent set (similarly,

we can use $\eta = u_{\mathbb{H}}$), to perform a summation over the variable Z , and again merge the D sumchecks in Equation (3) into one

$$\begin{aligned} & \sum_{j=0}^{D-1} \sum_{Z, Y, X \in \mathbb{H}} \eta(\alpha_2, Z) r(\alpha_1, Y) \hat{M}_j(Z, Y, X) \hat{w}_j(X) \\ &= \sum_{Z, X \in \mathbb{H}} \eta(\alpha_2, Z) r(\alpha_1, X) \left(\sum_{i=0}^{D-1} \hat{w}_i(X) L_{i, \mathbb{H}}(Z) \right), \end{aligned} \quad (5)$$

where α_2 is sampled by the verifier. Here we introduce Lagrange basis polynomial $L_{i, \mathbb{H}}(Z) = \prod_{j \in [D], j \neq i} \frac{\omega^j - Z}{\omega^j - \omega^i}$ s.t. $L_{i, \mathbb{H}}(Z) = 1$ iff $Z = \omega^i$ and 0 otherwise over $\mathbb{H}$. Thus, it holds that $\sum_{i=0}^{D-1} \hat{w}_i(X) L_{i, \mathbb{H}}(Z) = \hat{w}_i(X)$ for each $Z = \omega^i \in \mathbb{H}$, leading to Equation (5).

(5) Reduce Sumcheck Dimensions. A direct verification of Equation (5) would require $\mathcal{P}, \mathcal{V}$ to run a three-round sumcheck protocol that explicitly accounts for the summation dimensions X, Y , and Z . We next aim to avoid the direct summations over the dimensions Y, Z to reduce computational overhead.

RHS. We first simplify the RHS of Equation (5) by using the following observation

$$\sum_{Z \in \mathbb{H}} \eta(\alpha_2, Z) \left(\sum_{i=0}^{D-1} \hat{w}_i(X) L_{i, \mathbb{H}}(Z) \right) = \sum_{i=0}^{D-1} \hat{w}_i(X) \underbrace{\left(\sum_{Z \in \mathbb{H}} \eta(\alpha_2, Z) L_{i, \mathbb{H}}(Z) \right)}_{\eta'_{i, \alpha_2}},$$

where $\eta'_{i, \alpha_2} = \sum_{Z \in \mathbb{H}} \eta(\alpha_2, Z) L_{i, \mathbb{H}}(Z) = \eta(\alpha_2, z_i)$ for all $i \in [D]$ and $z_i = \omega^i$. The RHS of Equation (5) now turns to

$$\sum_{X \in \mathbb{H}} r(\alpha_1, X) \left(\sum_{i=0}^{D-1} \eta'_{i, \alpha_2} \hat{w}_i(X) \right),$$

which only requires a summation over a single dimension X .

LHS. To avoid direct summations over Y, Z of the LHS, we follow and extend the sparse matrix arithmetization technique from Fractal [16] by folding the kernels $\eta(\alpha_2, \cdot)$ and $r(\alpha_1, \cdot)$ into a new tensor polynomial. We first let

$$F_j(X) = \sum_{Z, Y \in \mathbb{H}} \eta(\alpha_2, Z) r(\alpha_1, Y) \hat{M}_j(Z, Y, X). \quad (6)$$

We now derive a new low-degree object whose single evaluation at variable X agrees with $F_j(X)$. First, we expand $\hat{M}_j$ in the Lagrange basis over $\mathbb{H}$

$$\hat{M}_j(Z, Y, X) = \sum_{a, b, c \in \mathbb{H}} L_{i_c, \mathbb{H}}(Z) L_{i_b, \mathbb{H}}(Y) L_{i_a, \mathbb{H}}(X) \mathbf{M}_j[c, b, a], \quad (7)$$

where $\mathbf{M}_j[c, b, a]$ denotes the tensor entry of $\mathbf{M}_j$ at $(Z, Y, X) = (c, b, a) \in \mathbb{H}^3$. Substituting Equation (7) into Equation (6), we obtain

$$\begin{aligned} F_j(X) &= \sum_{a, b, c \in \mathbb{H}} \left(\sum_{Z \in \mathbb{H}} \eta(\alpha_2, Z) L_{i_c, \mathbb{H}}(Z) \right) \\ &\quad \left(\sum_{Y \in \mathbb{H}} r(\alpha_1, Y) L_{i_b, \mathbb{H}}(Y) \right) L_{i_a, \mathbb{H}}(X) \mathbf{M}_j[c, b, a]. \end{aligned}$$

By construction, we have $u_{\mathbb{H}}(X, a) = u_{\mathbb{H}}(a, a) L_{i_a, \mathbb{H}}(X)$ for $a = \omega^{ia} \in \mathbb{H}$. Thus, setting $r = \eta = u_{\mathbb{H}}$, we have

$$r(\alpha_1, b) = u_{\mathbb{H}}(b, b) L_{i_b, \mathbb{H}}(\alpha_1), \quad \eta(\alpha_2, c) = u_{\mathbb{H}}(c, c) L_{i_c, \mathbb{H}}(\alpha_2).$$

Hence we obtain

$$F_j(X) = \sum_{a,b,c \in \mathbb{H}} L_{i_a, \mathbb{H}}(X) L_{i_b, \mathbb{H}}(\alpha_1) L_{i_c, \mathbb{H}}(\alpha_2) \mathbf{M}_j[c, b, a] u_{\mathbb{H}}(b, b) u_{\mathbb{H}}(c, c).$$

Then we introduce a new tensor $\mathbf{M}_j^*$ on $\mathbb{H}^3$ s.t.

$$\mathbf{M}_j^*[a, b, c] = \mathbf{M}_j[c, b, a] \cdot u_{\mathbb{H}}(b, b) \cdot u_{\mathbb{H}}(c, c).$$

That is, $\mathbf{M}_j^*$ is obtained from $\mathbf{M}_j$ by permuting the three coordinates from (c, b, a) to (a, b, c) and multiplying by the diagonal factors on the two absorbed coordinates. Let $\hat{M}_j^*(X, Y, Z)$ denote its low-degree extension where

$$\hat{M}_j^*(X, Y, Z) = \sum_{a,b,c \in \mathbb{H}} L_{i_a, \mathbb{H}}(X) L_{i_b, \mathbb{H}}(Y) L_{i_c, \mathbb{H}}(Z) \mathbf{M}_j^*[a, b, c].$$

Then Equation (6) can be written as

$$F_j(X) = \hat{M}_j^*(X, \alpha_1, \alpha_2).$$

Therefore, the double summation over Y and Z are re-expressed as a single evaluation of the new low-degree tensor polynomial $\hat{M}_j^*$ on point α_1 and α_2 . We can now translate this construction directly back into the sparse representation. By evaluating the last two variables of $\hat{M}_j^*$ at (α_1, α_2) , we obtain

$$\begin{aligned} \hat{M}_j^*(X, \alpha_1, \alpha_2) &= \sum_{k \in \mathbb{K}} u_{\mathbb{H}}(X, \hat{\text{col}}_j(k)) u_{\mathbb{H}}(\alpha_1, \hat{\text{row}}_j(k)) u_{\mathbb{H}}(\alpha_2, \hat{\text{mat}}_j(k)) \hat{\text{val}}_j^*(k), \end{aligned} \quad (8)$$

where $\hat{\text{val}}_j^*(k) = \hat{\text{val}}_j(k) u_{\mathbb{H}}(\hat{\text{row}}_j(k), \hat{\text{row}}_j(k)) u_{\mathbb{H}}(\hat{\text{mat}}_j(k), \hat{\text{mat}}_j(k))$. Equivalently, as $\hat{\text{row}}_j(k), \hat{\text{mat}}_j(k) \in \mathbb{H}$, we rewrite Equation (8) as

$$\hat{M}_j^*(X, \alpha_1, \alpha_2) = \sum_{k \in \mathbb{K}} \frac{Z_{\mathbb{H}}(X) Z_{\mathbb{H}}(\alpha_1) Z_{\mathbb{H}}(\alpha_2) \hat{\text{val}}_j^*(k)}{(X - \hat{\text{col}}_j(k))(\alpha_1 - \hat{\text{row}}_j(k))(\alpha_2 - \hat{\text{mat}}_j(k))}.$$

Consequently, the sumcheck claim becomes

$$\sum_{X \in \mathbb{H}} \left(r(\alpha_1, X) \left(\sum_{i=0}^{D-1} \eta'_{i, \alpha_2} \hat{w}_i(X) \right) - \sum_{j=0}^{D-1} \hat{M}_j^*(X, \alpha_1, \alpha_2) \hat{w}_j(X) \right) = 0. \quad (9)$$

(6) Reduce Verifier Cost for Sumcheck. We apply the univariate sumcheck protocol from Aurora [7] to prove Equation (9). To achieve a sublinear verifier, we perform two rounds of sumcheck.

Round 1. Sumcheck for Equation (9). The prover finds a quotient polynomial $Q_1(X)$ s.t.

$$\begin{aligned} Q_1(X) Z_{\mathbb{H}}(X) + X R_1(X) &= r(\alpha_1, X) \left(\sum_{i=0}^{D-1} \eta'_{i, \alpha_2} \hat{w}_i(X) \right) - \sum_{j=0}^{D-1} \hat{M}_j^*(X, \alpha_1, \alpha_2) \hat{w}_j(X). \end{aligned}$$

The verifier samples a random $\beta_1 \leftarrow \mathbb{F}$ and checks whether the above equation holds at the random $X = \beta_1$.

As directly evaluating $\sum_{j=0}^{D-1} \hat{M}_j^*(\beta_1, \alpha_1, \alpha_2) \hat{w}_j(\beta_1)$ incurs an $O(N)$ computational cost for $\mathcal{V}$, we let $\mathcal{P}$ compute this result, denoted by

$$\sigma = \sum_{j=0}^{D-1} \sum_{k \in \mathbb{K}} \frac{Z_{\mathbb{H}}(\beta_1) \cdot Z_{\mathbb{H}}(\alpha_1) \cdot Z_{\mathbb{H}}(\alpha_2) \cdot \hat{\text{val}}_j^*(k)}{(\beta_1 - \hat{\text{col}}_j(k))(\alpha_1 - \hat{\text{row}}_j(k))(\alpha_2 - \hat{\text{mat}}_j(k))} \cdot \hat{w}_j(\beta_1), \quad (10)$$

and invoke the sumcheck protocol again to prove its correctness.

Round 2. Sumcheck for Equation (10). Note that Equation (10) is a rational sumcheck over the domain $\mathbb{K}$. To perform this check, we use auxiliary polynomials to reduce it to a traditional sumcheck and a well-formation check. For each $j \in [D]$, we use $a_j(X)$ and $b_j(X)$ to denote the numerator and denominator polynomials of the rational expression, i.e.,

$$\begin{aligned} a_j(X) &= Z_{\mathbb{H}}(\beta_1) \cdot Z_{\mathbb{H}}(\alpha_1) \cdot Z_{\mathbb{H}}(\alpha_2) \cdot \hat{w}_j(\beta_1) \cdot \hat{\text{val}}_j^*(X), \\ b_j(X) &= (\beta_1 - \hat{\text{col}}_j(X))(\alpha_1 - \hat{\text{row}}_j(X))(\alpha_2 - \hat{\text{mat}}_j(X)). \end{aligned}$$

We define $f_j(X)$ that agrees with $\frac{a_j(X)}{b_j(X)}$ for each $X \in \mathbb{K}$. Then following existing techniques [7], Equation (10) can be reduced to

$$\sigma = \sum_{j=0}^{D-1} \sum_{X \in \mathbb{K}} f_j(X) = |\mathbb{K}| \sum_{j=0}^{D-1} f_j(0).$$

Additionally, $\mathcal{P}$ has to prove the well-formation of $f_j(X)$, i.e., $f_j(X) = \frac{a_j(X)}{b_j(X)}$ for all $X \in \mathbb{K}$. Here we use a random linear combination to batch check all $f_j(X)$ for $j \in [D]$. Specifically, $\mathcal{V}$ samples and sends a random challenge γ . $\mathcal{P}$ then computes $Q_2(X)$ s.t.

$$\sum_{j=0}^{D-1} \gamma^j (a_j(X) - b_j(X) \cdot f_j(X)) = Q_2(X) Z_{\mathbb{K}}(X).$$

The verifier then samples a random $\beta_2 \leftarrow \mathbb{F}$ and checks whether the above equation holds at $X = \beta_2$. Upon successful verification of Round-2 checks, the verifier substitutes σ into the Round-1 sumcheck equation to complete the first sumcheck.

The verifier can check the above equations with $O(\sqrt{N})$ cost setting $B = 3C$, and thus achieves sublinear complexity. Using Wasp^S as the underlying SIMD ZKP, we obtain Wasp^C, and the overall verifier cost remains sublinear. A formal description of Wasp^C and its security proof are provided in Appendix F.

6 Evaluation

6.1 Implementation

We instantiate the AHE primitive using the BGV scheme [11], and implement our PCS and the Wasp system in C++. We adopt some key optimizations to improve practical efficiency, as summarized below (additional details are provided in Appendix G).

Inner-product Computation. RLWE-based AHE schemes support SIMD-style plaintext packing, allowing multiple messages to be processed in parallel within a single ciphertext. However, the inner-product computations in our PCS involve cross-slot operations that require expensive ciphertext rotations. We adopt two methods to address this: (1) inner-product with transposed encoding, which encodes witnesses as polynomials and computes inner products via homomorphic linear combinations without online rotations; (2) inner-product with polynomial reversal, which leverages the cyclic convolution property of the quotient ring $\mathbb{Z}_q[x]/(x^N + 1)$ to aggregate pairwise products into the constant term. Both methods reduce the online-phase overhead to B ciphertext–plaintext multiplications and B ciphertext additions for committing B polynomials.

Ciphertext Packing. When the batch size is smaller than the number of available plaintext slots, some slots will be unused and wasted. We employ a tree-like packing method that iteratively combines

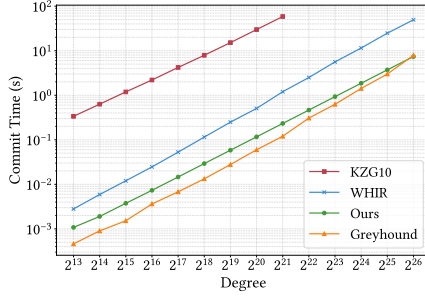


Figure 4: Commit time of PCS.

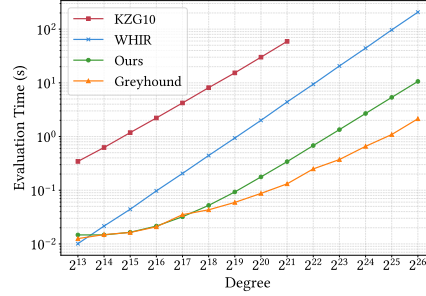


Figure 5: Evaluation time of PCS.

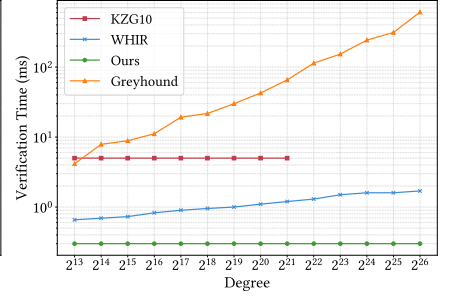


Figure 6: Verification time of PCS.

Table 2: Evaluation results of Wasp^S and the SIMD ZKP of Antman [42] on a SIMD circuit of a total size 2^{26} .

Protocol	Batch Size	$\mathcal{P}$ Time (s)	$\mathcal{V}$ Time (s)	Comm. (MB)
Wasp ^S	2^9	30.40 (1×)	0.044 (1×)	17.65 (1×)
	2^{10}	33.84 (1×)	0.022 (1×)	9.18 (1×)
	2^{11}	35.13 (1×)	0.012 (1×)	4.94 (1×)
	2^{12}	37.29	0.0065	2.82
	2^{13}	41.43	0.0040	1.77
Antman	2^9	42.73 (1.28×)	1.37 (31.14×)	16.95 (0.96×)
	2^{10}	43.38 (1.28×)	1.35 (61.36×)	8.49 (0.92×)
	2^{11}	48.00 (1.37×)	1.43 (120×)	4.27 (0.86×)

pairs of ciphertexts using negacyclic shifts and Galois automorphisms, folding all input ciphertexts into a single packed ciphertext over logarithmic levels. This technique significantly reduces the number of ciphertexts in the proof and shrinks the proof size.

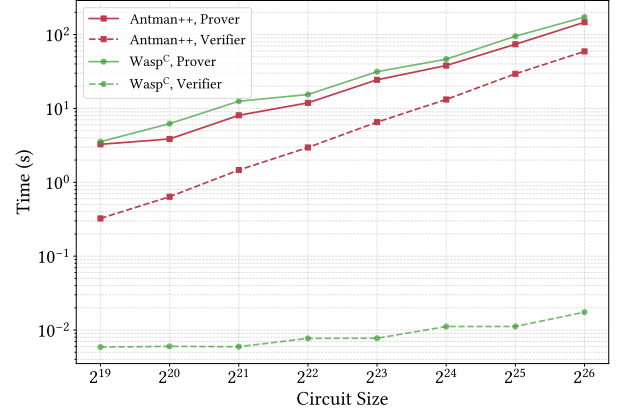
6.2 Experimental Setup

We work over a prime field $\mathbb{F}_p$ with a 59-bit modulus, the same as Antman(++). We target a computational security parameter $\kappa = 128$ and a statistical security parameter $\lambda > 40$. Since the 59-bit modulus p does not provide 128-bit computational security, we follow the traditional approach [29] to repeat the challenging phase sufficient times. We conduct four types of evaluations:

- We compare our PCS with the representative pairing-based PCS KZG10 [28], the SOTA coding-based PCS WHIR [4], and the lattice-based PCS Greyhound [34].
- We compare Wasp^S with the SIMD ZKP of Antman.
- We compare Wasp^C with the SIMD-to-general ZKP Antman++.
- We compare Wasp^G with the pairing-based zkSNARK Plonk [21], the lattice-based zkSNARK ISW21 [27], the coding-based zkSNARK DeepFold-HP [13, 25], and the VOLE-based ZKP Quick-silver [43].⁷

We run all the protocols on an Intel(R) Xeon(R) Platinum 8369B CPU @2.90GHz with 256 GB memory, using a single thread. We report the averaged results of multiple runs.

⁷For Plonk, we benchmark an optimized implementation over the BLS12-381 curve. DeepFold-HP is the scheme obtained by applying the DeepFold PCS to the PIOP system of HyperPlonk [13]. We didn't compare Wasp^G with the more recent lattice-based Luna [38], VOLE-based JesseQ [33] and Lin et al. [31] or coding-based zkSNARKs from STIR and WHIR [3, 4] due to a lack of ZKP implementations. We use a rate of 2 for coding-based PCS and zkSNARK.

Figure 7: Time cost of Antman++ and Wasp^C.

6.3 Performance of Our PCS

We show the evaluation results of PCS in Figure 4, 5 and 6. We didn't report the results of KZG10 for degrees larger than 2^{21} due to long running times.

Commit Time. Our PCS achieves notably higher commit efficiency than KZG10 and WHIR. It is $253 \sim 312\times$ faster than KZG10 across degrees from 2^{13} to 2^{21} , and $2.6 \sim 6.7\times$ faster than the SOTA coding-based WHIR. These significant efficiency gains of our PCS stem from the use of AHE operations instead of expensive multi-scalar multiplications required by KZG10 or massive coding and hash operations required by WHIR. Compared to the lattice-based Greyhound, our PCS is slightly slower in committing, but the gap shrinks as the degree increases. Concretely, our PCS is $2.34\times$ slower at degree 2^{13} , but is $1.1\times$ faster at degree 2^{26} .

Evaluation Time. Our PCS also shows significant improvements in evaluation efficiency over KZG10 and WHIR. Specifically, we achieve a speedup of $23.4 \sim 174\times$ over KZG10 across the tested degrees, and can be $19.5\times$ faster than WHIR at degree 2^{26} .

Compared to Greyhound, our PCS shows comparable evaluation efficiency at degrees less than 2^{18} , and is $1.6 \sim 5\times$ slower at degrees between $2^{19} \sim 2^{26}$. Nevertheless, the faster evaluation opening phase of Greyhound comes at the cost of a much slower verification phase, as we show below.

Verification Time. Our PCS shows a notable verifier efficiency with a constant 0.3 ms, $16.7\times$ faster than KZG10's 5 ms; this is because the AHE decryption operation is cheaper than the pairing operation. Compared to the logarithmic verifier of WHIR, our

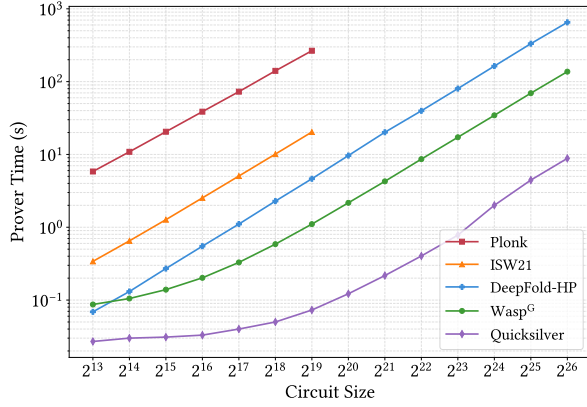


Figure 8: Prover time of different general ZKPs.

constant verifier achieves a $2.2 \sim 5.7\times$ speedup across the tested degrees. The lattice-based construction Greyhound has $O(\sqrt{N})$ verifier complexity, and our verifier can be $2034\times$ faster than that of Greyhound at degree 2^{26} .

Proof Size. The KZG10 scheme achieves the smallest proof size with only 81 bytes, while ours takes 121 KB. This overhead comes as a trade-off for our fast prover and verifier, since a ciphertext in our AHE scheme is larger than a single BLS12-381 group element. The polylogarithmic proof size of Greyhound grows from 31.6 KB at degree 2^{14} to 54.7 KB at degree 2^{26} , while WHIR’s proof size ranges from 72.8 KB to 214.3 KB.

6.4 Performance of Wasp^S

We summarize the evaluation results of the SIMD ZKPs in Table 2. We fix the circuit size to 2^{26} , and to see the effect of batch size B (or subcircuit size $C = N/B$) on the overall performance, we vary B from 2^9 to 2^{13} . We don’t have the results of Antman with batch size larger than 2^{11} as their source code doesn’t support $B > 2^{11}$.

Prover Time. Wasp^S achieves higher prover efficiency than Antman with factors ranging from $1.38 \sim 1.46\times$ across the tested batch sizes. This stems from our optimized scheme that avoids polynomial-degree doubling (i.e., Antman requires committing polynomials of degree $2B - 2$ while ours only requires degree $B - 1$).

Verifier Time. The verifier of Wasp^S is $31.14 \sim 120\times$ faster than that of Antman across different batch sizes, and we believe the gap will be more pronounced with larger batch sizes $B = 2^{12}$ and 2^{13} . This is mainly because our verifier is non-interactive, and the verifier cost gets lower as C decreases (c.f. Table 1).

Communication. The communication cost of Wasp^S is slightly higher than that of Antman, but overall they are comparable. This is because both communication costs are dominated by the $O(C)$ AHE ciphertexts, and their differences lie in different numbers of field elements (i.e., $O(C)$ of Wasp^S vs. $O(B)$ of Antman).

Effect of Batch Size. The proving time of both Wasp^S and Antman grows slightly as B increases, as they both require $O(N \log B)$ field operations. The verifier of Wasp^S requires $O(C)$ AHE decryption and $O(C)$ field operations, and thus the verifier time is reduced roughly linearly in B ; the verifier of Antman requires $O(C)$ AHE decryption plus $O(B)$ field operations, and more importantly, it

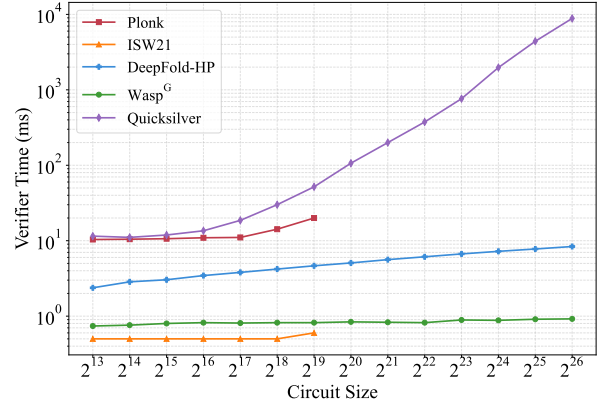


Figure 9: Verifier time of different general ZKPs.

is interactive, thus showing minimal fluctuation across the tested batch sizes. Considering communication, the costs of both protocols are roughly reduced linearly as B increases, since they are both dominated by $O(C)$ AHE ciphertexts.

6.5 Performance of Wasp^C

We show the evaluation results of the SIMD-to-general ZKPs in Figure 7. We use the same parameters for both protocols.⁸

Prover Time. The prover efficiency of Wasp^C is slightly lower than that of Antman++, with a factor between $1.1 \sim 1.6\times$ across the tested circuit sizes. This comes at a cost of achieving succinctness by delegating the matrix evaluation to the prover. Nevertheless, our new wire consistency protocol eliminates the need for redundant witness commitments required by Antman++, leading to a minor efficiency loss for the prover.

Verifier Time. Wasp^C achieves significant improvements over Antman++ in verifier efficiency, with factors ranging from $55\times$ at circuit size 2^{19} to $3,384\times$ at circuit size 2^{26} . This benefits from our optimized techniques for reducing the linear verifier cost to sublinear one, and also the fact that Wasp^C adopts the faster, non-interactive Wasp^S as the underlying SIMD ZKP.

Communication. The communication cost of Wasp^C is $1.77 \sim 1.88\times$ higher than that of Antman++ with the same parameters B, C . Specifically, Wasp^C’s proof size ranges from 2.71 MB at circuit size 2^{19} to 4.11 MB at circuit size 2^{24} , while Antman++’s ranges from 1.44 MB to 2.32 MB. This overhead mainly comes from our wire consistency check for committing extra polynomials, as a trade-off for achieving succinctness.⁹

6.6 Performance of Wasp^G

We show evaluation results of the general ZKPs in Figure 8 and 9. We didn’t report the results of Plonk and ISW21 at circuit sizes larger than 2^{19} due to memory exhaustion or long running times.

Prover Time. Wasp^G can achieve two orders of improvement in prover efficiency over Plonk. At circuit size 2^{16} , Wasp^G is $241\times$

⁸We set $B = 9$ for circuit size 2^{19} and 2^{20} , $B = 10$ for circuit size 2^{21} and 2^{22} , and $B = 11$ for circuit size 2^{23} and 2^{24} . For circuit size 2^{25} and 2^{26} , Wasp^G uses $B = 12$ while Antman++ uses $B = 11$ as its implementation doesn’t support $B > 2^{11}$.

⁹At larger circuit sizes, Wasp^C shows comparable or lower communication costs (e.g., 6.10 vs. 7.99 MB at size 2^{26}), as it uses a larger batch size B .

faster than Plonk in proving (1.1 vs. 265.6 s). Given that the two schemes adopt the same PIOP system, this substantial improvement implies the great advantage of our PCS over KZG10. Compared with the lattice-based zkSNARK ISW21, Wasp^G also demonstrates a notable improvement with 18.4× speedup at circuit size 2¹⁹. Compared with the coding-based zkSNARK DeepFold-HP, Wasp^G shows an improvement of 2.4 ~ 8.4× across the tested circuit sizes. When compared with the linear-time Quicksilver, Wasp^G is 3 ~ 22× slower in proving, mainly due to Quicksilver adopting the lightweight VOLE-based gate-by-gate proving paradigm. Nevertheless, Wasp^G offers succinct verification and constant proof size, which Quicksilver cannot provide, as shown below.

Verifier Time. Both Wasp^G, Plonk and ISW21 achieve constant verification time. Wasp^G maintains a verification time of 0.7 ~ 0.9 ms, while Plonk takes tens of milliseconds (10.4 ~ 19 ms) across the tested circuit sizes. The lattice-based construction ISW21 achieves a verification time at approximately 0.5 ms, slightly faster than ours. The coding-based zkSNARK DeepFold-HP shows a verification time at 2.4 ~ 8.4 ms. The non-succinct ZKP scheme Quicksilver exhibits linear verification time. Concretely, the verifier time grows from 14 ms at circuit size 2¹⁴ to 8.81 s at 2²⁶, which is roughly four orders of magnitude slower than ours.

Proof Size. Plonk achieves the most compact proofs with size only 2 KB, benefiting from the efficiency of KZG10 commitments over pairing-friendly curves. Our protocol incurs a proof size of 122 KB, larger than Plonk, but remains practical in real-world applications. ISW21 also features a constant proof size of 20 KB. The proof size of DeepFold-HP scales logarithmically, growing from 95.5 KB to 436.6 KB across the tested circuit sizes. Quicksilver, however, incurs a linear proof size, which reaches 4.33 MB at a small circuit size of 2¹³, and increases to 531.52 MB at a circuit size of 2²⁶, showing a more than 4,000× size expansion of ours.

Open Science

In line with open science principles, we support artifact evaluation by ensuring the availability, functionality, and reproducibility of our work. We provide a public repository containing the implementation of PCS and our proposed systems (Wasp^S, Wasp^G, and Wasp^C), available at <https://github.com/antcplab/openwasp>. For Antman and Antman++, which are not open-sourced, we re-implement their functionality based on the original papers and include these implementations in our repository. For other baselines, we use official open-source implementations and provide links and configuration details in the artifact.

Ethical Considerations

This work presents a new polynomial commitment scheme and a zero-knowledge protocol, which are general-purpose cryptographic primitives. As such, they do not involve the collection or use of personal data, nor do they interact with human subjects.

We acknowledge that zero-knowledge techniques can be considered dual-use technologies. While they enable strong privacy guarantees and are essential for protecting sensitive information, they may also be misused to conceal malicious or illegal activities.

Our work focuses on improving the efficiency and security of these primitives without targeting any specific application domain.

We emphasize that the deployment of such cryptographic techniques should be accompanied by appropriate regulatory and system-level safeguards. We do not provide application-specific tools or systems that could directly facilitate misuse. We believe that the benefits of advancing privacy-preserving technologies outweigh the potential risks when used responsibly.

Generative AI Usage

AI tools (including ChatGPT-5.4 by OpenAI and Claude Opus 4.6) were used for grammar and language polishing, and also to assist our implementation. All technical ideas, theoretical derivations, system design, and implementation results were not generated by AI. All AI-assisted content was carefully reviewed and revised. The authors take full responsibility for the final version of the paper.

References

- [1] Martin R. Albrecht, Giacomo Fenzi, Oleksandra Lapiha, and Ngoc Khanh Nguyen. 2024. SLAP: Succinct Lattice-Based Polynomial Commitments from Standard Assumptions. In *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part VI (Lecture Notes in Computer Science)*, Marc Joye and Gregor Leander (Eds.). Springer, 90–119. doi:10.1007/978-3-031-58754-2_4
- [2] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2017. Liger: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM, 2087–2104. doi:10.1145/3133956.3134104
- [3] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. 2024. STIR: Reed-Solomon Proximity Testing with Fewer Queries. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X (Lecture Notes in Computer Science)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, 380–413. doi:10.1007/978-3-031-68403-6_12
- [4] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. 2025. WHIR: Reed-Solomon proximity testing with super-fast verification. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 214–243.
- [5] Carsten Baum, Lennart Braun, Cyprien Delpéch de Saint Guilhem, Michael Klooß, Emmanuela Orsini, Lawrence Roy, and Peter Scholl. 2023. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In *Annual international cryptography conference*. Springer, 581–615.
- [6] Carsten Baum, Alex J. Malozemoff, Marc B. Rosen, and Peter Scholl. 2021. Mac'n'Cheese: Zero-Knowledge Proofs for Boolean and Arithmetic Circuits with Nested Disjunctions. In *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part IV (Lecture Notes in Computer Science)*, Tal Malkin and Chris Peikert (Eds.). Springer, 92–122. doi:10.1007/978-3-030-84259-8_4
- [7] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. 2019. Aurora: Transparent Succinct Arguments for R1CS. In *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 11476)*, Yuval Ishai and Vincent Rijmen (Eds.). Springer, 103–128. doi:10.1007/978-3-030-17653-2_4
- [8] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. [n. d.]. Semi-homomorphic Encryption and Multiparty Computation. In *EUROCRYPT 2011*.
- [9] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. 2013. Succinct Non-interactive Arguments via Linear Interactive Proofs. In *Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3-6, 2013. Proceedings (Lecture Notes in Computer Science)*, Amit Sahai (Ed.). Springer, 315–333. doi:10.1007/978-3-642-36594-2_18
- [10] Ellette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. 2018. Compressing Vector OLE. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 896–912. doi:10.1145/3243734.3243868

- [11] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM TOCT* (2014).
- [12] Dung Bui, Haotian Chu, Geoffroy Couteau, Xiao Wang, Chenkai Weng, Kang Yang, and Yu Yu. 2025. An Efficient ZK Compiler from SIMD Circuits to General Circuits. *J. Cryptol.* 38, 1 (2025), 10. doi:10.1007/S00145-024-09531-4
- [13] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 499–530.
- [14] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadeh Riazi. 2020. {SANNs}: Scaling up secure approximate {k-Nearest} neighbors search. In *29th USENIX Security Symposium (USENIX Security 20)*. 2111–2128.
- [15] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39*. Springer, 738–768.
- [16] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. 2020. Fractal: Post-quantum and Transparent Recursive Proofs from Holography. In *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I (Lecture Notes in Computer Science)*, Anne Canteaut and Yuval Ishai (Eds.). Springer, 769–793. doi:10.1007/978-3-030-45721-1_27
- [17] Samuel Dittmer, Yuval Ishai, Steve Lu, and Rafail Ostrovsky. 2022. Improving Line-Point Zero Knowledge: Two Multiplications for the Price of One. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (Los Angeles, CA, USA) (CCS '22)*. Association for Computing Machinery, New York, NY, USA, 829–841. doi:10.1145/3548606.3559385
- [18] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. 2021. Line-Point Zero Knowledge and Its Applications. In *2nd Conference on Information-Theoretic Cryptography, ITC 2021, Virtual Conference, July 23-26, 2021 (LIPICs)*, Stefano Tessaro (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 5:1–5:24. doi:10.4230/LIPICs.ITC.2021.5
- [19] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.
- [20] Ariel Gabizon and Zachary J. Williamson. 2021. fflonk: a Fast-Fourier inspired verifier efficient version of PlonK. *IACR Cryptol. ePrint Arch.* (2021), 1167. <https://eprint.iacr.org/2021/1167>
- [21] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019).
- [22] S Goldwasser, S Micali, and C Rackoff. 1985. The knowledge complexity of interactive proof-systems. In *Proceedings of the seventeenth annual ACM symposium on Theory of computing*. 291–304.
- [23] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S Wahby. 2023. Brakedown: Linear-time and field-agnostic SNARKs for RICS. In *Annual International Cryptology Conference*. Springer, 193–226.
- [24] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*. Springer, 305–326.
- [25] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. 2025. DeepFold: Efficient Multilinear Polynomial Commitment from Reed-Solomon Code and Its Application to Zero-knowledge Proofs. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujio Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 3497–3516. <https://www.usenix.org/conference/usenixsecurity25/presentation/guo-yanpei>
- [26] Intak Hwang, Jinyeong Seo, and Yongsoo Song. 2024. Concretely Efficient Lattice-Based Polynomial Commitment from Standard Assumptions. In *Advances in Cryptology – CRYPTO 2024*, Leonid Reyzin and Douglas Stebila (Eds.). Springer Nature Switzerland, Cham, 414–448.
- [27] Yuval Ishai, Hang Su, and David J Wu. 2021. Shorter and faster post-quantum designated-verifier zkSNARKs from lattices. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 212–234.
- [28] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*. Springer, 177–194.
- [29] Marcel Keller, Emmanuela Orsini, and Peter Scholl. 2016. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 830–842.
- [30] Marcel Keller, Valerio Pastro, and Dragos Rotaru. 2018. Overdrive: Making SPDZ great again. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 158–189.
- [31] Fuchun Lin, Chaoping Xing, and Yizhou Yao. 2024. Interactive Line-Point Zero-Knowledge with Sublinear Communication and Linear Computation. In *Advances in Cryptology – ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings. Part V (Kolkata, India)*. Springer-Verlag, Berlin, Heidelberg, 337–366. doi:10.1007/978-981-96-0935-2_11
- [32] Helger Lipmaa, Roberto Parisella, and Janno Siim. 2024. Constant-size zk-SNARKs in ROM from falsifiable assumptions. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 34–64.
- [33] Mengling Liu, Yang Heng, Xingye Lu, and Man Ho Au. 2025. JesseQ: Efficient Zero-Knowledge Proofs for Circuits Over Any Field. In *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*, Marina Blanton, William Enck, and Cristina Nita-Rotaru (Eds.). IEEE, 3402–3420. doi:10.1109/SP61157.2025.00203
- [34] Ngoc Khanh Nguyen and Gregor Seiler. 2024. Greyhound: Fast Polynomial Commitments from Lattices. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X (Lecture Notes in Computer Science)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, 243–275. doi:10.1007/978-3-031-68403-6_8
- [35] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Shehank Burra. [n. d.]. A New Approach to Practical Active-Secure Two-Party Computation. In *CRYPTO 2011*.
- [36] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. 2013. Signatures of correct computation. In *Theory of Cryptography Conference*. Springer, 222–242.
- [37] Jacob T Schwartz. 1980. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM (JACM)* (1980).
- [38] Ron Steinfield, Amin Sakzad, Muhammed F. Esgin, Veronika Kuchta, Mert Yassi, and Raymond K. Zhao. 2024. LUNA: Quasi-Optimally Succinct Designated-Verifier Zero-Knowledge Arguments from Lattices. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM, 3167–3181. doi:10.1145/3658644.3670345
- [39] Hoeteck Wee and David J. Wu. 2023. Succinct Vector, Polynomial, and Functional Commitments from Lattices. In *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part III (Lecture Notes in Computer Science)*, Carmit Hazay and Martijn Stam (Eds.). Springer, 385–416. doi:10.1007/978-3-031-30620-4_13
- [40] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. 2021. Wolverine: Fast, Scalable, and Communication-Efficient Zero-Knowledge Proofs for Boolean and Arithmetic Circuits. In *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 1074–1091. doi:10.1109/SP40001.2021.00056
- [41] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, Michael D. Bailey and Rachel Greenstadt (Eds.). USENIX Association, 501–518. <https://www.usenix.org/conference/usenixsecurity21/presentation/weng>
- [42] Chenkai Weng, Kang Yang, Zhaomin Yang, Xiang Xie, and Xiao Wang. 2022. AntMan: Interactive Zero-Knowledge Proofs with Sublinear Communication. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 2901–2914. doi:10.1145/3548606.3560667
- [43] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. 2021. QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 2986–3001. doi:10.1145/3460120.3484556
- [44] Yibin Yang, Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, and Tal Rabin. 2025. Gold OPRF: Post-Quantum Oblivious Power-Residue PRF. In *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*, Marina Blanton, William Enck, and Cristina Nita-Rotaru (Eds.). IEEE, 259–278. doi:10.1109/SP61157.2025.00116
- [45] Hadas Zeilberger, Binyi Chen, and Ben Fisch. 2024. BaseFold: efficient field-agnostic polynomial commitment schemes from foldable codes. In *Annual International Cryptology Conference*. Springer, 138–169.
- [46] Richard Zippel. 1979. Probabilistic algorithms for sparse polynomials. In *International symposium on symbolic and algebraic manipulation*.

A Additional Preliminaries

A.1 Univariate Sumcheck Protocol

We review the univariate sumcheck protocol from Aurora [7]. Let $\mathbb{F}$ be a field, and let $\mathbb{H} \subset \mathbb{F}$ be a multiplicative subgroup of size $|\mathbb{H}|$. Define vanishing polynomial:

$$Z_{\mathbb{H}}(X) = \prod_{a \in \mathbb{H}} (X - a) = X^{|\mathbb{H}|} - 1.$$

Let $f(X) \in \mathbb{F}[X]$ be a polynomial with $\deg(f) < d$. Then $\sum_{X \in \mathbb{H}} f(X) = \sigma$ iff there exist unique polynomials $Q(X), R(X) \in \mathbb{F}[X]$ such that

$$f(X) = X \cdot R(X) + \frac{\sigma}{|\mathbb{H}|} + Q(X) Z_{\mathbb{H}}(X),$$

with

$$\deg(R) < |\mathbb{H}| - 1, \quad \deg(Q) < d - |\mathbb{H}|.$$

In particular, if $\deg(f) < |\mathbb{H}|$, then $Q = 0$ and

$$\sum_{X \in \mathbb{H}} f(X) = |\mathbb{H}| \cdot f(0).$$

A.2 Sparse Matrix Arithmetization of Fractal

Let $\vec{r} = (u_{\mathbb{H}}(a, Y))_{a \in \mathbb{H}} \in \mathbb{F}[Y]^{\mathbb{H}}$ denote a vector comprising linearly independent polynomials in Y . Let $u_{\mathbb{M}}(X, Y)$ be the low-degree extension of the product $\vec{r}\mathbf{M} \in \mathbb{F}[Y]^{\mathbb{H}}$. To efficiently evaluate $u_{\mathbb{M}}(X, Y)$ at a challenge point sampled uniformly from $\mathbb{F} \times \mathbb{F}$, Fractal utilizes a sparse matrix arithmetization, specifically by considering a transformed matrix $\mathbf{M}^*$ derived from $\mathbf{M}$. Let $M_{a,b}$ be the entry $\mathbf{M}[a, b]$. We review Claim 6.7 of Fractal [16] below.

LEMMA 1 (CLAIM 6.7 OF FRACTAL [16]). *For any matrix $\mathbf{M} \in \mathbb{F}^{\mathbb{H} \times \mathbb{H}}$, define the auxiliary matrix $\mathbf{M}^* \in \mathbb{F}^{\mathbb{H} \times \mathbb{H}}$ such that its entries are given by $M_{a,b}^* := M_{b,a} \cdot u_{\mathbb{H}}(b, b)$ for all $a, b \in \mathbb{H}$. It follows that $\|\mathbf{M}^*\| = \|\mathbf{M}\|$, and the following identity holds:*

$$u_{\mathbf{M}}(X, Y) \equiv \hat{\mathbf{M}}^*(X, Y).$$

PROOF. Starting from the definition of a low-degree extension, denote $u_{\mathbf{M}}(X, Y)$ as the following expansion over the Lagrange basis $\{L_{a,\mathbb{H}}(X)\}_{a \in \mathbb{H}}$:

$$\begin{aligned} u_{\mathbf{M}}(X, Y) &\equiv \sum_{a \in \mathbb{H}} (\vec{r}\mathbf{M})_a \cdot L_{a,\mathbb{H}}(X) \\ &\equiv \sum_{a \in \mathbb{H}} L_{a,\mathbb{H}}(X) \left(\sum_{b \in \mathbb{H}} \mathbf{M}_{b,a} \cdot u_{\mathbb{H}}(b, Y) \right) \end{aligned}$$

Utilizing the property that $u_{\mathbb{H}}(b, Y) \equiv u_{\mathbb{H}}(b, b) L_{b,\mathbb{H}}(Y)$, we can substitute this into the inner summation:

$$u_{\mathbf{M}}(X, Y) = \sum_{a \in \mathbb{H}} \sum_{b \in \mathbb{H}} L_{a,\mathbb{H}}(X) L_{b,\mathbb{H}}(Y) \cdot \mathbf{M}_{b,a} u_{\mathbb{H}}(b, b) \equiv \hat{\mathbf{M}}^*(X, Y). \quad \square$$

B Assumptions for AHE

We state the security assumptions on additively homomorphic encryption required for proving the security of our protocol. First, the AHE scheme is assumed to be *CPA-secure* and *circuit-private*. CPA security guarantees that the encrypted powers $\{\langle \Lambda^i \rangle\}_{i=1}^d$ hide the secret evaluation point Λ , while circuit privacy hides the affine computation used to derive a ciphertext. We denote the corresponding distinguishing advantages by $\epsilon_{\text{CPA}}(\kappa)$ and $\epsilon_{\text{CP}}(\kappa)$.

We utilize *linear targeted malleability* [9], as in AntMan, and further specialize it to the plaintext vector $(\Lambda, \Lambda^2, \dots, \Lambda^d)$. Namely, for any PPT adversary producing polynomially many valid target ciphertexts $\tilde{C}_1, \dots, \tilde{C}_k$, there exists a PPT simulator, not given Λ , whose output induces polynomials

$$p_1, \dots, p_k \in \mathbb{F}_{\leq d}[X]$$

such that their joint decryption behavior is computationally indistinguishable from

$$p_1(\Lambda), \dots, p_k(\Lambda).$$

Each polynomial has degree at most d and can be written as

$$p_j(X) = a_{j,0} + \sum_{\ell=1}^d a_{j,\ell} X^\ell,$$

where all coefficients $\{a_{j,\ell}\}_{j,\ell}$ are jointly determined without knowledge of Λ . We refer to this specialization as the *polynomial linear targeted malleability* property. We denote the corresponding distinguishing advantage by $\epsilon_{\text{PLTM}}(\kappa)$. Following the analysis [30], the BGV-AHE scheme is regarded as a candidate instantiation.

C Formal Descriptions of Our PCS and Batch Scheme

We give a formal description of our PCS and its batch scheme in Protocol C.1 and Protocol C.2, respectively.

Our PCS security analysis follows the approach of [32]. We first prove binding. Recall that committing to $f \in \mathbb{F}_{\leq d}[X]$ produces an IT-PAC of the hidden evaluation $f(\Lambda)$: the prover holds the tag M_f , while the verifier holds (K_f, Δ, Λ) , and

$$M_f = K_f + \Delta f(\Lambda).$$

We prove the PCS properties directly from this authentication relation.

LEMMA 2 (HIDDEN-POINT ROOT BOUND). *Assume that the AHE scheme is CPA-secure. Let $\Lambda \leftarrow \mathbb{F}$ be uniform, and suppose that a PPT adversary, given $\{\langle \Lambda^i \rangle\}_{i=1}^d$ and no additional information about Λ , outputs a nonzero polynomial $h \in \mathbb{F}_{\leq D}[X]$. Then*

$$\Pr[h(\Lambda) = 0] \leq \epsilon_{\text{HP}}(\kappa) := \epsilon_{\text{CPA}}(\kappa) + \frac{D}{|\mathbb{F}|}.$$

More generally, if the adversary outputs N such polynomials in one execution, then

$$\Pr[\exists i : h_i \neq 0 \wedge h_i(\Lambda) = 0] \leq \epsilon_{\text{HP}}^{(N)}(\kappa) := \epsilon_{\text{CPA}}(\kappa) + \frac{ND}{|\mathbb{F}|}.$$

PROOF. By CPA security, we replace the encrypted powers by encryptions of fixed values, at distinguishing cost $\epsilon_{\text{CPA}}(\kappa)$. In the resulting hybrid, the polynomials output by the adversary are independent of Λ . The claim then follows from the Schwartz–Zippel bound and a union bound over the N root events. $\square$

THEOREM 1 (BINDING OF Π_{PCS}). *For every PPT adversary outputting distinct polynomials $f, g \in \mathbb{F}_{\leq d}[X]$, the probability that the same IT-PAC (M_f, K_f) authenticates both $f(\Lambda)$ and $g(\Lambda)$ is bounded by*

$$\Pr[f \neq g \wedge M_f = K_f + \Delta f(\Lambda) \wedge M_f = K_f + \Delta g(\Lambda)] \leq \epsilon_{\text{bind}},$$

PROTOCOL C.1. Π_{PCS} (Our Polynomial Commitment Scheme)

Let $\text{AHE} = (\text{Setup}, \text{Gen}, \text{Enc}, \text{Dec})$ be an additively homomorphic encryption scheme. W.l.o.g., let d be the degree of the polynomials to be committed.

- **Setup(1^κ)**: on input the security parameter κ ,
 - (1) $\mathcal{P}, \mathcal{V}$ send (init) to $\mathcal{F}_{\text{VOLE}}$, and $\mathcal{V}$ receives a random $\Delta \in \mathbb{F}$ from it.
 - (2) $\mathcal{P}, \mathcal{V}$ initialize the AHE parameters by running $\text{pp} = \text{AHE.Setup}(1^\kappa)$ and $\text{sk} = \text{AHE.Gen}(\text{pp})$. $\mathcal{V}$ picks a random $\Lambda \in \mathbb{F}$, and runs $\langle \Lambda^i \rangle = \text{AHE.Enc}(\text{sk}, \Lambda^i; r_i)$ with a random r_i for each $i \in [d]$. $\mathcal{V}$ outputs $\text{srs} = (\langle \Lambda \rangle, \dots, \langle \Lambda^d \rangle)$ as the SRS.
- **Commit(srs, f)**: on input srs and a polynomial $f \in \mathbb{F}_{\leq d}[X]$,
 - (1) $\mathcal{P}, \mathcal{V}$ send (extend, 1) to $\mathcal{F}_{\text{VOLE}}$ to get a VOLE correlation m_f^*, k_f^*, r_f s.t. $m_f^* = k_f^* + \Delta \cdot r_f$.
 - (2) $\mathcal{P}$ computes $\langle s_f \rangle = f_0 + \sum_{i=1}^d f_i \langle \Lambda^i \rangle - r_f$ where $\{f_i\}_{i=0}^d$ are the coefficients of f . $\mathcal{P}$ sends the ciphertext $\langle s_f \rangle$ to $\mathcal{V}$.
 - (3) $\mathcal{V}$ decrypts $s_f = \text{AHE.Dec}(\text{sk}, \langle s_f \rangle)$. $\mathcal{P}, \mathcal{V}$ obtain a commitment $[f(\Lambda)]$ by setting $M_f = m_f^*$ and $K_f = k_f^* - \Delta \cdot s_f$ respectively.
- **Open(srs, C, f)**: on input srs , a commitment C , and a polynomial $f \in \mathbb{F}_{\leq d}[X]$,
 - (1) $\mathcal{V}$ computes $f(\Lambda)$.
 - (2) Given M_f, k_f w.r.t. $C = [f(\Lambda)]$, $\mathcal{V}$ checks if $M_f = K_f + \Delta \cdot f(\Lambda)$ holds, then outputs $b = 0$ if the check passes, and $b = 1$ otherwise.
- **Eval(srs, f, r)**: on input srs , a polynomial $f \in \mathbb{F}_{\leq d}[X]$, and an evaluation point r ,
 - (1) $\mathcal{P}$ computes $q(X) = (f(X) - f(r))/(X - r) \in \mathbb{F}_{\leq d-1}[X]$.
 - (2) $\mathcal{P}, \mathcal{V}$ send (extend, 1) to $\mathcal{F}_{\text{VOLE}}$ to get a VOLE correlation m_q^*, k_q^*, r_q s.t. $m_q^* = k_q^* + \Delta \cdot r_q$. $\mathcal{P}$ sets $M_q = m_q^*$.
 - (3) $\mathcal{P}$ computes $\langle s_q \rangle = q_0 + \sum_{i=1}^{d-1} q_i \langle \Lambda^i \rangle - r_q$, where $\{q_i\}_{i=0}^{d-1}$ are the coefficients of q , and $\langle M_t \rangle = (\langle \Lambda \rangle - r)M_q - M_f$. $\mathcal{P}$ outputs $\pi = (\langle s_q \rangle, \langle M_t \rangle)$ as the evaluation proof.
- **Verify(srs, C, r, y, π)**: on input a commitment C , evaluation point r , claimed evaluation y , and proof π ,
 - (1) $\mathcal{V}$ decrypts $M_t = \text{AHE.Dec}(\text{sk}, \langle M_t \rangle)$, $s_q = \text{AHE.Dec}(\text{sk}, \langle s_q \rangle)$, and sets $K_q = k_q^* - \Delta \cdot s_q$.
 - (2) $\mathcal{V}$ computes $K_t = (\Lambda - r) \cdot K_q - (K_f + \Delta \cdot y)$, then outputs $b = 1$ if $M_t = K_t$, and $b = 0$ otherwise.

PROTOCOL C.2. $\Pi_{\text{PCS-Batch}}$ (Our Batch Polynomial Commitment Scheme)

Let $\text{AHE} = (\text{Setup}, \text{Gen}, \text{Enc}, \text{Dec})$ be an additively homomorphic encryption scheme. W.l.o.g., let d be the maximum degree of the polynomials to be committed. The batch scheme supports proving evaluations of n polynomials $\{f_i\}_{i=1}^n$ at a point r .

- **Setup(1^κ)**: Same as Π_{PCS} .
- **Commit($\text{srs}, \{f_i\}_{i=1}^n$)**: on input srs and n polynomials $\{f_i \in \mathbb{F}_{\leq d}[X]\}_{i=1}^n$,
 - (1) $\mathcal{P}, \mathcal{V}$ send (extend, n) to $\mathcal{F}_{\text{VOLE}}$ to get n VOLE correlations $\{m_{f_i}^*, k_{f_i}^*, r_{f_i}\}_{i=1}^n$ s.t. $m_{f_i}^* = k_{f_i}^* + \Delta \cdot r_{f_i}$.
 - (2) For each $i \in [n]$, $\mathcal{P}$ computes $\langle s_{f_i} \rangle = f_{i,0} + \sum_{j=1}^d f_{i,j} \langle \Lambda^j \rangle - r_{f_i}$ where $\{f_{i,j}\}_{j=0}^d$ are the coefficients of f_i . $\mathcal{P}$ sends the ciphertexts $\{\langle s_{f_i} \rangle\}_{i=1}^n$ to $\mathcal{V}$.
 - (3) $\mathcal{V}$ decrypts $s_{f_i} = \text{AHE.Dec}(\text{sk}, \langle s_{f_i} \rangle)$ for all $i \in [n]$. $\mathcal{P}, \mathcal{V}$ obtain commitments $\{[f_i(\Lambda)]\}_{i=1}^n$ by setting $M_{f_i} = m_{f_i}^*$ and $K_{f_i} = k_{f_i}^* - \Delta \cdot s_{f_i}$ respectively.
- **Open($\text{srs}, \{C_i\}_{i=1}^n, \{f_i\}_{i=1}^n$)**: on input srs , commitments $\{C_i\}_{i=1}^n$, and polynomials $\{f_i\}_{i=1}^n$,
 - (1) $\mathcal{V}$ computes $f_i(\Lambda)$.
 - (2) Given M_{f_i}, K_{f_i} w.r.t. $C_i = [f_i(\Lambda)]$, $\mathcal{V}$ checks if $M_{f_i} = K_{f_i} + \Delta \cdot f_i(\Lambda)$ holds for all $i \in [n]$. $\mathcal{V}$ outputs $b = 0$ if all the checks pass, and $b = 1$ otherwise.
- **BatchEval($\text{srs}, \{f_i\}_{i=1}^n, r, \{y_i\}_{i=1}^n$)**: on input srs , n polynomials $\{f_i\}_{i=1}^n$, an evaluation point r ,
 - (1) $\mathcal{V}$ sends a fresh random batching challenge $\rho \leftarrow \mathbb{F}$ to $\mathcal{P}$.
 - (2) $\mathcal{P}$ computes a quotient polynomial $q(X) = \sum_{i=1}^n \rho^{i-1} \cdot (f_i(X) - f_i(r))/(X - r)$, where $\deg(q) \leq d - 1$.
 - (3) $\mathcal{P}, \mathcal{V}$ send (extend, 1) to $\mathcal{F}_{\text{VOLE}}$ to get a VOLE correlation m_q^*, k_q^*, r_q s.t. $m_q^* = k_q^* + \Delta \cdot r_q$. $\mathcal{P}$ sets $M_q = m_q^*$.
 - (4) $\mathcal{P}$ computes $\langle s_q \rangle = q_0 + \sum_{i=1}^{d-1} q_i \langle \Lambda^i \rangle - r_q$ where $\{q_i\}_{i=0}^{d-1}$ are the coefficients of q .
 - (5) $\mathcal{P}$ computes $\langle M_t \rangle = M_q \cdot (\langle \Lambda \rangle - r) - \sum_{i=1}^n \rho^{i-1} M_{f_i}$. $\mathcal{P}$ outputs $\pi = (\langle s_q \rangle, \langle M_t \rangle)$ as the batch evaluation proof.
- **BatchVerify($\text{srs}, \{C_i\}_{i=1}^n, r, \{y_i\}_{i=1}^n, \pi$)**: on input commitments, evaluation point r , claimed evaluations $\{y_i\}_{i=1}^n$, and proof π ,
 - (1) $\mathcal{V}$ decrypts $M_t = \text{Dec}(\text{sk}, \langle M_t \rangle)$, $s_q = \text{Dec}(\text{sk}, \langle s_q \rangle)$, and sets $K_q = k_q^* - \Delta \cdot s_q$.
 - (2) $\mathcal{V}$ computes $K_t = K_q \cdot (\Lambda - r) - \sum_{i=1}^n \rho^{i-1} (K_{f_i} + \Delta \cdot y_i)$, then outputs $b = 1$ if $M_t = K_t$, and $b = 0$ otherwise.

where

$$\varepsilon_{\text{pbind}} := \varepsilon_{\text{HP}}(\kappa) + \frac{1}{|\mathbb{F}|}.$$

If Δ is sampled from $\mathbb{F}^*$, the final term is omitted.

PROOF. If the same IT-PAC (M_f, K_f) authenticates two distinct polynomials $f, g \in \mathbb{F}_{\leq d}[X]$, then

$$\Delta(f(\Lambda) - g(\Lambda)) = 0.$$

Conditioned on $\Delta \neq 0$, the nonzero polynomial

$$h(X) := f(X) - g(X)$$

has degree at most d and satisfies $h(\Lambda) = 0$. By Lemma 2, this occurs with probability at most $\epsilon_{\text{HP}}(\kappa)$. Finally, if $\Delta \leftarrow \mathbb{F}$, $\Pr[\Delta = 0] = \frac{1}{|\mathbb{F}|}$. $\square$

THEOREM 2 (EVALUATION BINDING). *Assume that the AHE scheme satisfies polynomial linear targeted malleability and CPA security, and that the ideal VOLE functionality hides the global MAC key $\Delta \leftarrow \mathbb{F}$ from the prover. Then, for any PPT adversary,*

$$\Pr \left[\begin{array}{l} \text{Verify}(C, r, y_0, \pi_0) = 1, \\ \text{Verify}(C, r, y_1, \pi_1) = 1, \\ y_0 \neq y_1 \end{array} \right] \leq \epsilon_{\text{ebind}},$$

where

$$\epsilon_{\text{ebind}} := \epsilon_{\text{PLTM}}(\kappa) + \epsilon_{\text{HP}}^{(2)}(\kappa) + \frac{1}{|\mathbb{F}|}.$$

PROOF. Fix a commitment $C = (M_f, K_f)$ and two accepted openings (r, y_b, π_b) , $b \in \{0, 1\}$, where

$$\pi_b = (\langle s_{q_b} \rangle, \langle M_{t_b} \rangle).$$

We first move to the ideal experiment guaranteed by polynomial linear targeted malleability, incurring distinguishing loss $\epsilon_{\text{PLTM}}(\kappa)$. The target ciphertexts

$$\langle s_f \rangle, \quad \{\langle s_{q_b} \rangle, \langle M_{t_b} \rangle\}_{b \in \{0, 1\}}$$

admit joint low-degree explanations determined without knowledge of Λ .

Combining these explanations with the corresponding VOLE relations, there exist jointly determined polynomials

$$f^*, q_b^*, t_b^* \in \mathbb{F}_{\leq d}[X]$$

such that

$$M_f - K_f = \Delta f^*(\Lambda), \quad M_{q_b} - K_{q_b} = \Delta q_b^*(\Lambda), \quad M_{t_b} = t_b^*(\Lambda).$$

Since the b -th opening is accepted,

$$M_{t_b} = (\Lambda - r)K_{q_b} - (K_f + \Delta y_b).$$

Substitution gives

$$A_b(\Lambda) = \Delta H_b(\Lambda),$$

where

$$A_b(X) := t_b^*(X) - (X - r)M_{q_b} + M_f$$

and

$$H_b(X) := f^*(X) - y_b - (X - r)q_b^*(X).$$

By the privacy of the ideal VOLE functionality, conditioned on the prover's view and on Λ , the MAC key Δ remains uniform. If $H_b(\Lambda) \neq 0$, the acceptance equation uniquely determines

$$\Delta = \frac{A_b(\Lambda)}{H_b(\Lambda)}.$$

Hence, if either $H_0(\Lambda)$ or $H_1(\Lambda)$ is nonzero, simultaneous acceptance occurs with probability at most $1/|\mathbb{F}|$. Therefore, except with this probability,

$$H_0(\Lambda) = H_1(\Lambda) = 0.$$

The polynomials H_0, H_1 have degree at most d and are jointly determined without knowledge of Λ . By Lemma 2,

$$\Pr[\exists b \in \{0, 1\} : H_b \neq 0 \wedge H_b(\Lambda) = 0] \leq \epsilon_{\text{HP}}^{(2)}(\kappa).$$

Thus, except with the stated error,

$$H_b(X) \equiv 0 \quad (b = 0, 1).$$

Evaluating at $X = r$ gives

$$y_b = f^*(r), \quad b = 0, 1,$$

and hence $y_0 = y_1$, a contradiction. $\square$

THEOREM 3 (($d+1$)-SPECIAL SOUNDNESS). *Let*

$$\text{tr}_i = (M_f, K_f, r_i, y_i, \pi_i), \quad i \in [d+1],$$

be accepting transcripts with the same IT-PAC (M_f, K_f) and pairwise distinct points r_i . Let $\hat{f} \in \mathbb{F}_{\leq d}[X]$ be the polynomial interpolating $\{(r_i, y_i)\}_{i \in [d+1]}$. Then, except with probability

$$\epsilon_{\text{ss}} := \epsilon_{\text{PLTM}}(\kappa) + \epsilon_{\text{HP}}^{(d+1)}(\kappa) + \frac{1}{|\mathbb{F}|},$$

we have

$$M_f = K_f + \Delta \hat{f}(\Lambda)$$

and $\hat{f}(r_i) = y_i$ for all $i \in [d+1]$.

PROOF. We move to the ideal experiment guaranteed by polynomial linear targeted malleability, incurring loss $\epsilon_{\text{PLTM}}(\kappa)$. The commitment ciphertext and all ciphertexts appearing in the $d+1$ evaluation proofs admit joint low-degree explanations determined without knowledge of Λ . In particular, there exists $f^* \in \mathbb{F}_{\leq d}[X]$ such that

$$M_f - K_f = \Delta f^*(\Lambda).$$

Applying the same argument as in the proof of evaluation binding to all $d+1$ accepting openings, except with probability

$$\frac{1}{|\mathbb{F}|} + \epsilon_{\text{HP}}^{(d+1)}(\kappa),$$

we obtain

$$y_i = f^*(r_i) \quad \text{for all } i \in [d+1].$$

Since the points r_i are pairwise distinct and both f^* and $\hat{f}$ have degree at most d , uniqueness of interpolation implies

$$\hat{f} = f^*.$$

Therefore

$$M_f = K_f + \Delta \hat{f}(\Lambda),$$

as required. $\square$

We now lift $(d+1)$ -special soundness to black-box extractability, following the rewinding approach of [32]. The extractor is given rewinding access to the prover state after the commitment is fixed and before the challenge is sampled.

LEMMA 3 (REWINDING LEMMA). *Fix the prover state immediately after the IT-PAC (M_f, K_f) is fixed, and regard the prover's random tape as part of this state. Let the initial challenge $r_0 \leftarrow \mathbb{F}$ be uniform. There exists a black-box rewinding procedure that, whenever the initial transcript is accepting, attempts to obtain d additional accepting transcripts with pairwise distinct challenges. Then*

$$\Pr[\text{initial transcript accepts but rewinding fails}] \leq \frac{d}{|\mathbb{F}|}.$$

Moreover, the expected number of rewind prover executions is at most d .

PROOF. Fix the prover state and let m be the number of challenges $r \in \mathbb{F}$ for which the resulting evaluation proof is accepting.

If $m \leq d$, rewinding may fail to collect d additional accepting challenges, but the initial challenge is accepting with probability $m/|\mathbb{F}| \leq d/|\mathbb{F}|$. This gives the failure bound.

For the running time, after an accepting initial challenge, the extractor samples the remaining challenges uniformly without replacement. If $m \geq d + 1$, the expected number of samples required to obtain d additional accepting challenges is $\frac{d|\mathbb{F}|}{m}$. Since the initial challenge is accepted with probability $m/|\mathbb{F}|$, its unconditional contribution to the expected number of rewind executions is at most d . If $m \leq d$, the extractor makes at most $|\mathbb{F}| - 1$ rewind executions, whose unconditional contribution is at most

$$\frac{m}{|\mathbb{F}|} (|\mathbb{F}| - 1) \leq d.$$

Thus, for every fixed prover state, the expected number of rewind executions is at most d . $\square$

THEOREM 4 (BLACK-BOX EXTRACTABILITY OF Π_{PCS}). *Let $\mathcal{P}^*$ be a stateful PPT prover that fixes the IT-PAC (M_f, K_f) before receiving a uniform evaluation challenge $r_0 \leftarrow \mathbb{F}$. Then there exists an expected-PPT extractor $\mathcal{E}^{\mathcal{P}^*}$ with black-box rewinding access to $\mathcal{P}^*$ such that*

$$\Pr \left[\begin{array}{l} \text{Verify}(M_f, K_f, r_0, y_0, \pi_0) = 1 \\ \wedge \mathcal{E}^{\mathcal{P}^*} \text{ fails to output } \hat{f} \in \mathbb{F}_{\leq d}[X] \\ \text{with } M_f = K_f + \Delta \hat{f}(\Lambda) \end{array} \right] \leq \epsilon_{\text{ext}},$$

where

$$\epsilon_{\text{ext}} := \frac{d}{|\mathbb{F}|} + \epsilon_{\text{ss}}.$$

For every transcript (r_i, y_i, π_i) used in the extraction, the extracted polynomial satisfies

$$\hat{f}(r_i) = y_i.$$

PROOF. The extractor stores the prover state immediately before the evaluation challenge and obtains the initial transcript for $r_0 \leftarrow \mathbb{F}$. If the transcript is rejecting, it aborts.

Otherwise, the extractor applies Lemma 3: it rewinds to the stored state and samples fresh challenges uniformly without replacement until it obtains d additional accepting transcripts with distinct challenges, or no challenges remain.

Except with probability $d/|\mathbb{F}|$, this yields $d + 1$ accepting transcripts

$$(M_f, K_f, r_i, y_i, \pi_i), \quad i \in [d + 1],$$

with pairwise distinct r_i . The extractor interpolates the pairs (r_i, y_i) to obtain the unique $\hat{f} \in \mathbb{F}_{\leq d}[X]$, so by construction

$$\hat{f}(r_i) = y_i \quad \text{for all } i \in [d + 1].$$

By Theorem 3, except with probability ϵ_{ss} ,

$$M_f = K_f + \Delta \hat{f}(\Lambda).$$

A union bound therefore gives

$$\epsilon_{\text{ext}} \leq \frac{d}{|\mathbb{F}|} + \epsilon_{\text{ss}}.$$

Finally, Lemma 3 bounds the expected number of rewind prover executions by d , so the extractor runs in expected polynomial time. $\square$

D Security Proof of Wasp^S

In this section, we analyze the security of Wasp^S, beginning with a lemma for batch PCS.

LEMMA 4 (BATCH OPENING CONSISTENCY). *Let $f_1, \dots, f_m$ be the polynomials associated with m fixed commitments, and let $y_1, \dots, y_m$ be their claimed evaluations at a common point r . Suppose that the batch challenge $\xi \leftarrow \mathbb{F}$ is sampled after all claims y_j are fixed. If the batched opening is accepting, then, except with probability*

$$\frac{m-1}{|\mathbb{F}|},$$

we have

$$y_j = f_j(r) \quad \text{for all } j \in [m].$$

PROOF. An accepting batched opening implies

$$\sum_{j=1}^m \xi^{j-1} f_j(r) = \sum_{j=1}^m \xi^{j-1} y_j.$$

Let

$$E(T) := \sum_{j=1}^m (f_j(r) - y_j) T^{j-1}.$$

If some $y_j \neq f_j(r)$, then E is a nonzero polynomial of degree at most $m-1$. Since ξ is uniform and sampled after the claims are fixed, the Schwartz–Zippel bound gives

$$\Pr[E(\xi) = 0] \leq \frac{m-1}{|\mathbb{F}|}.$$

Hence, except with this probability, all claimed evaluations are correct. $\square$

THEOREM 5 (KNOWLEDGE SOUNDNESS OF Wasp^S). *Assume black-box extractability and batch opening consistency of the PCS for all commitments appearing in Protocol 3.1. Let $\epsilon_{\text{PCS}}(\kappa)$ bound their joint failure probability. Suppose that $\eta \xleftarrow{\$} \mathbb{F}$ is sampled after all wire-polynomial commitments are fixed, and that $r \xleftarrow{\$} \mathbb{F} \setminus \mathbb{H}$ is sampled after C_Q is fixed. Then a prover that makes the verifier accept while at least one of the B extracted executions violates a multiplication gate succeeds with probability at most*

$$\epsilon_{\text{PCS}}(\kappa) + \frac{C-1}{|\mathbb{F}|} + \frac{2B-2}{|\mathbb{F}| - B}.$$

All AHE and VOLE/PAC errors underlying PCS extraction and opening consistency are absorbed into $\epsilon_{\text{PCS}}(\kappa)$.

PROOF. Condition on successful PCS extraction and batch opening consistency. We obtain degree- $(B-1)$ polynomials f_j, g_j, h_j for the multiplication constraints and a degree- $(B-2)$ polynomial Q , all consistent with the accepted openings. Since the wire commitments are fixed before η is sampled, the polynomials

$$E_j(X) := f_j(X)g_j(X) - h_j(X)$$

are fixed independently of η .

Work in the quotient ring

$$\mathcal{R} := \mathbb{F}[X]/(Z_{\mathbb{H}}(X)).$$

If some extracted multiplication gate is false on $\mathbb{H}$, then at least one residue $\bar{E}_j \in \mathcal{R}$ is nonzero. Hence

$$A(T) := \sum_{j=0}^{C-1} T^j \bar{E}_j \in \mathcal{R}[T]$$

is nonzero and has degree at most $C - 1$. Choose an $\mathbb{F}$ -linear functional $\ell : \mathcal{R} \rightarrow \mathbb{F}$ that is nonzero on one nonzero coefficient of A . Then $\ell(A(T))$ is a nonzero polynomial over $\mathbb{F}$ of degree at most $C - 1$. Therefore,

$$\Pr_{\eta \leftarrow \mathbb{F}} \left[\sum_{j=0}^{C-1} \eta^j E_j(X) \equiv 0 \pmod{Z_{\mathbb{H}}(X)} \right] \leq \frac{C-1}{|\mathbb{F}|}. \quad (\text{A.1})$$

Condition on the complement of this event. Then

$$S(X) := \sum_{j=0}^{C-1} \eta^j E_j(X)$$

is not divisible by $Z_{\mathbb{H}}(X)$. After η is fixed, the prover commits to Q , and hence

$$P(X) := Q(X)Z_{\mathbb{H}}(X) - S(X)$$

is a nonzero polynomial fixed before r is sampled. Since $\deg Z_{\mathbb{H}} = B$,

$$\deg P \leq 2B - 2.$$

By PCS extraction and Lemma 4, every value used in the final verification equation equals the evaluation at r of its corresponding extracted polynomial. Thus acceptance implies

$$P(r) = 0.$$

Since r is uniform over $\mathbb{F} \setminus \mathbb{H}$,

$$\Pr[P(r) = 0] \leq \frac{2B-2}{|\mathbb{F}| - B}. \quad (\text{A.2})$$

Combining the PCS failure probability with (A.1) and (A.2) proves the claim. $\square$

E Formal Description of Wasp^G

The PIOP system of our Wasp^G protocol adopts that of the original Plonk [21]. We now give a full description of Wasp^G below (more details please refer to [21]).

Setup: Given an arithmetic circuit of size N , the algorithm runs $\text{PCS.Setup}(1^\kappa)$ with supported degree N , and generates a universal structured reference string srs . It also derives the selector coefficients and permutation information:

$$\text{srs}, (q_{Mi}, q_{Li}, q_{Ri}, q_{Oi}, q_{Ci})_{i=0}^{N-1}, (\sigma_0, \dots, \sigma_{3N-1}).$$

Let $\mathbb{H} = \langle \omega \rangle \subseteq \mathbb{F}^*$ be the size- N evaluation domain, let $Z_{\mathbb{H}}(X) = X^N - 1$, and choose distinct coset constants $k_0 = 1, k_1, k_2$. These values encode the circuit constraints (selector coefficients) and the wiring (permutation σ).

Selector polynomials:

$$\begin{aligned} q_M(X) &= \sum_{i=0}^{N-1} q_{Mi} L_i(X), & q_L(X) &= \sum_{i=0}^{N-1} q_{Li} L_i(X), \\ q_R(X) &= \sum_{i=0}^{N-1} q_{Ri} L_i(X), & q_O(X) &= \sum_{i=0}^{N-1} q_{Oi} L_i(X), \\ q_C(X) &= \sum_{i=0}^{N-1} q_{Ci} L_i(X). \end{aligned}$$

These polynomials encode the gate constraints of the circuit in interpolated form.

Permutation polynomials:

$$\begin{aligned} S_L(X) &= \sum_{i=0}^{N-1} \sigma(i) L_i(X), & S_R(X) &= \sum_{i=0}^{N-1} \sigma(N+i) L_i(X), \\ S_O(X) &= \sum_{i=0}^{N-1} \sigma(2N+i) L_i(X). \end{aligned}$$

These polynomials describe how wires are connected across different gates. The prover then runs $\text{PCS.Commit}(\text{srs}, \cdot)$ to generate commitments:

$$C_{q_M}, C_{q_L}, C_{q_R}, C_{q_O}, C_{q_C}, C_{S_L}, C_{S_R}, C_{S_O}.$$

Let $\text{PI}(X)$ denote the public-input polynomial appearing in the PLONK gate identity.

Prove:

Round 1: $\mathcal{P}$ computes wire polynomials $w_L(X), w_R(X), w_O(X)$ and invokes $\text{PCS.Commit}(\text{srs}, \cdot)$ to generate commitment C_L, C_R, C_O .

After these commitments are fixed, $\mathcal{V}$ samples $\beta, \gamma \xleftarrow{\$} \mathbb{F}$ and sends them to $\mathcal{P}$.

Round 2: $\mathcal{P}$ computes the PLONK grand-product polynomial P .

$$R_j = \frac{\prod_{i=0}^2 (w_{i \cdot N+j} + \beta k_i \omega^j + \gamma)}{\prod_{i=0}^2 (w_{i \cdot N+j} + \sigma(i \cdot N+j) \beta + \gamma)},$$

$$P(X) = L_1(X) + \sum_{i=1}^{N-1} L_{i+1}(X) \prod_{j=1}^i R_j.$$

The prover commits to P , obtaining C_P . After C_P is fixed, $\mathcal{V}$ samples $\alpha \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathcal{P}$.

Round 3: $\mathcal{P}$ computes quotient polynomial. It follows that a divisibility relation holds:

$$\begin{aligned} t(X) &= \frac{1}{Z_{\mathbb{H}}(X)} \left[w_L(X) w_R(X) q_M(X) + w_L(X) q_L(X) + w_R(X) q_R(X) \right. \\ &\quad \left. + w_O(X) q_O(X) + \text{PI}(X) + q_C(X) \right. \\ &\quad \left. + \alpha \left((w_L(X) + \beta X + \gamma)(w_R(X) + \beta k_1 X + \gamma) \right. \right. \\ &\quad \left. \left. (w_O(X) + \beta k_2 X + \gamma) P(X) \right. \right. \\ &\quad \left. \left. - (w_L(X) + \beta S_L(X) + \gamma)(w_R(X) + \beta S_R(X) + \gamma) \right. \right. \\ &\quad \left. \left. (w_O(X) + \beta S_O(X) + \gamma) P(X\omega) \right) \right. \\ &\quad \left. + \alpha^2 (P(X) - 1) L_1(X) \right]. \end{aligned}$$

The quotient polynomial $t(X)$ has degree $3N-1$. To enable a unified commitment and opening, decompose $t(X)$ into polynomials $t_{lo}(X)$, $t_{mid}(X)$, and $t_{hi}(X)$ of degree at most N , such that

$$t(X) = t_{lo}(X) + X^N t_{mid}(X) + X^{2N} t_{hi}(X).$$

The prover commits to the three quotient chunks, obtaining $C_{t_{lo}}, C_{t_{mid}}, C_{t_{hi}}$. The verifier samples $r \xleftarrow{\$} \mathbb{F} \setminus \mathbb{H}$.

Round 4: $\mathcal{P}$ evaluates all required m polynomials at point r :

$$y_i := f_i(r), \quad i = 0, \dots, m-1,$$

where

$$\{f_i\}_{i=0}^{m-1} = \{w_L, w_R, w_O, P, S_L, S_R, S_O, q_L, q_R, q_O, q_C, t_{lo}, t_{mid}, t_{hi}\},$$

$\mathcal{P}$ generates a batched opening proof:

$$\pi = \text{BatchEval}(\text{srs}, \{f_i\}_{i=0}^{m-1}, r, \{y_i\}_{i=0}^{m-1}).$$

All commitments and proofs can be packed into a single ciphertext C_t and sent to the verifier.

Verify: The verifier $\mathcal{V}$ holds:

Preprocessed input:

$$C_{q_M}, C_{q_L}, C_{q_R}, C_{q_O}, C_{q_C}, C_{S_L}, C_{S_R}, C_{S_O}.$$

Online input:

$$C_L, C_R, C_O, C_P, C_{t_{lo}}, C_{t_{mid}}, C_{t_{hi}}, \pi,$$

and evaluations $\{y_i\}_{i=0}^{m-1}$.

- (1) $\mathcal{V}$ uses the challenges β, γ, α, r generated in transcript order above.
- (2) $\mathcal{V}$ computes domain values:

$$Z_H(r), \quad L_1(r).$$

- (3) $\mathcal{V}$ verifies all openings:

$$\text{BatchVerify}\left(\text{srs}, \{C_{f_i}\}_{i=0}^{m-1}, r, \{y_i\}_{i=0}^{m-1}, \pi\right) \stackrel{?}{=} 1.$$

The verifier also checks

$$\text{PCS.Verify}(\text{srs}, C_P, \omega r, y_{P,\omega}, \pi_{P,\omega}) = 1.$$

- (4) $\mathcal{V}$ checks quotient identity at r :

$$\begin{aligned} & w_L(r)w_R(r)q_M(r) + w_L(r)q_L(r) + w_R(r)q_R(r) + w_O(r)q_O(r) \\ & + \text{PI}(r) + q_C(r) \\ & + \alpha \left[(w_L(r) + \beta r + \gamma) \right. \\ & \quad \cdot (w_R(r) + \beta k_1 r + \gamma) \\ & \quad \cdot (w_O(r) + \beta k_2 r + \gamma) P(r) \\ & \quad - (w_L(r) + \beta S_L(r) + \gamma) \\ & \quad \cdot (w_R(r) + \beta S_R(r) + \gamma) \\ & \quad \cdot (w_O(r) + \beta S_O(r) + \gamma) y_{P,\omega} \left. \right] \\ & + \alpha^2 (P(r) - 1) L_1(r) \\ & \stackrel{?}{=} Z_H(r) \left(t_{lo}(r) + r^N t_{mid}(r) + r^{2N} t_{hi}(r) \right). \end{aligned}$$

THEOREM 6 (KNOWLEDGE SOUNDNESS OF Wasp^G). *Let R be the circuit-satisfaction relation. Assume that the underlying PLONK PIOP for R is knowledge sound with soundness error $\varepsilon_{\text{Plonk}}$, and that the PCS provides black-box extractability and batch opening consistency for all polynomial commitments and openings appearing in the protocol, with failure probability ε_{PCS} . Then Wasp^G has soundness error at most*

$$\varepsilon_{\text{Plonk}} + \varepsilon_{\text{PCS}}.$$

PROOF. Condition on successful PCS black-box extraction and batch opening consistency. The extractor obtains degree-bounded polynomials

$$\widehat{w}_L, \widehat{w}_R, \widehat{w}_O, \widehat{P}, \widehat{t}_{lo}, \widehat{t}_{mid}, \widehat{t}_{hi}$$

corresponding to all online commitments. Every value accepted in the batched opening at r equals the evaluation of its corresponding extracted polynomial.

Since the challenges are sampled in the prescribed order— β, γ after the wire commitments, α after C_P , and r after the quotient commitments—the extracted polynomials together with these challenges define an accepting execution of the underlying PLONK PIOP.

By the assumed knowledge soundness of the PLONK PIOP, except with probability $\varepsilon_{\text{Plonk}}$, a witness $\mathbf{w}$ satisfying

$$(\mathbf{x}, \mathbf{w}) \in R$$

can be extracted. Accounting for the PCS extraction and opening-consistency failure gives the claimed bound. $\square$

F Formal Description and Knowledge Soundness of Wasp^C

We give the SIMD-to-general protocol in Protocol F.1 and analyze the security below.

THEOREM 7 (KNOWLEDGE SOUNDNESS OF Wasp^C). *Assume that the PCS is black-box extractable for every commitment appearing in the protocol and satisfies batch opening consistency. Let $\varepsilon_{\text{PCS}}^{\text{all}}$ bound the probability that any required PCS extraction or batch-opening consistency event fails. Let $\varepsilon_{\text{SIMD}}$ be the conditional soundness error of the underlying Wasp^S protocol defined after Theorem 5.*

Then, from any accepting execution of Wasp^C, there exists an expected-PPT black-box extractor that outputs a witness satisfying the padded general circuit, except with probability at most

$$\begin{aligned} \varepsilon_{\text{ks}} & \leq \varepsilon_{\text{PCS}}^{\text{all}} + \varepsilon_{\text{SIMD}} + \frac{2(D-1)}{|\mathbb{F}| - D} + \frac{2D-2}{|\mathbb{F}| - D} \\ & \quad + \frac{D-1}{|\mathbb{F}|} + \frac{4B-4}{|\mathbb{F}|}. \end{aligned}$$

PROOF. Condition on successful extraction of all PCS commitments required by the protocol and on all batch-opening consistency events. We obtain the degree-bounded polynomials

$$\widehat{w}_0, \dots, \widehat{w}_{D-1}, \quad Q_1, R, \quad f_0, \dots, f_{D-1}, \quad Q_2,$$

and every value accepted in a PCS opening equals the evaluation of its corresponding extracted polynomial.

By Theorem 5, conditioned on these PCS events, except with probability $\varepsilon_{\text{SIMD}}$, the values

$$\widehat{w}_j(k), \quad j < 3C, \quad k \in \mathbb{K},$$

PROTOCOL F.1. $\Pi_{\text{ZK}}^{\text{Compiler}}$ (The SIMD-to-General ZK Protocol Wasp^C)

Let C be a circuit over $\mathbb{F}$, and C' be a padded circuit of C using dummy gates with a total size of $N = BC$, such that C' can be split into B identical subcircuits each of size C . $\mathcal{P}$ holds B satisfying assignments (witnesses) to the B SIMD subcircuits. Let $D = \max(B, 3C)$ and w.l.o.g., let $\mathbb{H}$ be a multiplicative subgroup of $\mathbb{F}$ with order D and generator ω . Let $\mathbb{K}$ be a subgroup of $\mathbb{F}$ with order B . Let $Z_{\mathbb{H}}(X) = X^D - 1$ and $Z_{\mathbb{K}}(X) = X^B - 1$ be the vanishing polynomials over $\mathbb{H}$ and $\mathbb{K}$, respectively. Let PCS be the PCS described in $\Pi_{\text{PCS-Batch}}$ (Protocol C.2). Let ZKP be the SIMD ZKP described in $\Pi_{\text{ZK}}^{\text{SIMD}}$ (Protocol 3.1).

- **Public Parameters:** A sparse matrix $\mathbf{M} \in \mathbb{F}^{3N \times 3N}$ representing the wire connections of the circuit, with at most $3N$ non-zero entries. The matrix is partitioned into $3C \times 3C$ submatrices $\{\mathbf{M}_{ij}\}_{i,j \in [B]}$, each of size $B \times B$.
- **Setup:**
 - (1) Let $u_{\mathbb{H}}(X, Y) = (Z_{\mathbb{H}}(X) - Z_{\mathbb{H}}(Y))/(X - Y)$ be the formal derivative of the vanishing polynomial $Z_{\mathbb{H}}$.
 - (2) For each $\mathbf{M}_{ij}$, define functions $\text{row}_j, \text{col}_j, \text{mat}_j, \text{val}_j : \mathbb{K} \rightarrow \mathbb{F}$ mapping each $k \in \mathbb{K}$ to the row index, column index of non-zero entries, as well as the value $1/(u_{\mathbb{H}}(\text{row}_j(k), \text{row}_j(k))u_{\mathbb{H}}(\text{col}_j(k), \text{col}_j(k))u_{\mathbb{H}}(\text{mat}_j(k), \text{mat}_j(k)))$, where mat encodes index i . Let $\hat{\text{row}}_j, \hat{\text{col}}_j, \hat{\text{val}}_j, \hat{\text{mat}}_j$ be their low-degree extensions.
 - (3) Define the bivariate polynomial encoding: $\hat{M}_j(X, Y, Z) = \sum_{\kappa \in \mathbb{K}} u_{\mathbb{H}}(X, \hat{\text{row}}_j(\kappa)) \cdot u_{\mathbb{H}}(Y, \hat{\text{col}}_j(\kappa)) \cdot u_{\mathbb{H}}(Z, \hat{\text{mat}}_j(\kappa)) \cdot \hat{\text{val}}_j(\kappa)$.
 - (4) $\mathcal{P}, \mathcal{V}$ run $\text{PCS.Setup}(1^\kappa)$ with the supported degree $d = \max(B - 1, 3C - 1)$.
- **Prove:**
 - (1) **Subcircuit Satisfaction:** $\mathcal{P}, \mathcal{V}$ run the SIMD ZK protocol $\text{ZKP.Prove}(\cdot)$ to prove the satisfaction of the SIMD subcircuits.
 - (2) **Wire Consistency Check:** For each $i \in [C]$, let $\tilde{l}_i, \tilde{r}_i, \tilde{o}_i \in \mathbb{F}^B$ be all the wire values on the left, right, and output wires of this gate in the B subcircuits. Let $\tilde{\mathbf{w}} = (\{\tilde{l}_i, \tilde{r}_i, \tilde{o}_i\}_{i \in [C]}) \in \mathbb{F}^{3BC}$, and $\{\tilde{\mathbf{w}}_i^T\}_{i \in [3C]}$ be $3C$ partitioned length- B subvectors. Let $\hat{\mathbf{w}}_i(X)$ be the low-degree extension of the witness $\tilde{\mathbf{w}}_i^T$.
 - (a) $\mathcal{V}$ samples random challenges $\alpha_1, \alpha_2 \leftarrow \mathbb{F}$ and sends them to $\mathcal{P}$.
 - (b) $\mathcal{V}$ samples random coefficients $\{\eta_i\}_{i=0}^{D-1} \leftarrow \mathbb{F}^D$ and sends them to $\mathcal{P}$.
 - (c) Define the aggregated polynomial: $F_j(X) = \sum_{Z, Y \in \mathbb{H}} \eta(Z, Y) r(\alpha_1, Y) \hat{M}_j(Z, Y, X)$, where $r(X, Y) = \eta(X, Y) = u_{\mathbb{H}}(X, Y)$.
 - (d) **Round 1 (First Sumcheck):** $\mathcal{P}$ computes quotient polynomials $Q_1(X)$ and $R(X)$ such that $Q_1(X) Z_{\mathbb{H}}(X) + X R(X) = r(\alpha_1, X) \left(\sum_{i=0}^{D-1} \eta'_{i, \alpha_2} \hat{\mathbf{w}}_i(X) \right) - \sum_{j=0}^{D-1} \hat{M}_j^*(X, \alpha_1, \alpha_2) \hat{\mathbf{w}}_j(X)$, where $\eta'_{i, \alpha_2} = \eta(\alpha_2, z_i)$ for all $i \in [D]$ and $z_i = \omega^i$. $\mathcal{P}$ runs $\text{PCS.Commit}(\text{srs}, \cdot)$ to generate commitments C_{Q_1} and C_R .
 - (e) $\mathcal{V}$ samples a random $\beta_1 \leftarrow \mathbb{F}$ and sends it to $\mathcal{P}$.
 - (f) Let $G_1 = \{\{w_j\}_{j=0}^{D-1}, Q_1, R\}$ and $Y_1 = \{\{w_j(\beta_1)\}_{j=0}^{D-1}, Q_1(\beta_1), R(\beta_1)\}$. $\mathcal{P}$ runs $\pi_1 = \text{PCS.BatchEval}(\text{srs}, G_1, \beta_1)$ and sends (Y_1, π_1) to $\mathcal{V}$.
 - (g) $\mathcal{V}$ checks whether $Q_1(\beta_1) \cdot Z_{\mathbb{H}}(\beta_1) + \beta_1 R(\beta_1) \stackrel{?}{=} r(\alpha_1, \beta_1) \left(\sum_{i=0}^{D-1} \eta'_{i, \alpha_2} \hat{\mathbf{w}}_i(\beta_1) \right) - \sum_{j=0}^{D-1} \hat{M}_j^*(\beta_1, \alpha_1, \alpha_2) \hat{\mathbf{w}}_j(\beta_1)$.
 - (h) **Round 2 (Second Sumcheck):** To evaluate $\sum_{j=0}^{D-1} \hat{M}_j^*(\beta_1, \alpha_1, \alpha_2) \cdot \hat{\mathbf{w}}_j(\beta_1)$, compute
$$\sigma = \sum_{j=0}^{D-1} \sum_{k \in \mathbb{K}} \frac{Z_{\mathbb{H}}(\beta_1) \cdot Z_{\mathbb{H}}(\alpha_1) \cdot Z_{\mathbb{H}}(\alpha_2) \cdot \hat{\text{val}}_j^*(k)}{(\beta_1 - \hat{\text{col}}_j(k))(\alpha_1 - \hat{\text{row}}_j(k))(\alpha_2 - \hat{\text{mat}}_j(k))} \cdot \hat{\mathbf{w}}_j(\beta_1).$$
 - (i) Let $a_j(X) = Z_{\mathbb{H}}(\beta_1) \cdot Z_{\mathbb{H}}(\alpha_1) \cdot Z_{\mathbb{H}}(\alpha_2) \cdot w_j(\beta_1) \cdot \hat{\text{val}}_j(X)$, $b_j(X) = (\beta_1 - \hat{\text{col}}_j(X))(\alpha_1 - \hat{\text{row}}_j(X))(\alpha_2 - \hat{\text{mat}}_j(X))$. For every $j \in [D]$, $\mathcal{P}$ defines the unique polynomial $f_j \in \mathbb{F}_{<B}[X]$ such that $f_j(k) = a_j(k)/b_j(k)$ for every $k \in \mathbb{K}$. Equivalently, there exists a quotient polynomial $q_j(X)$ satisfying $a_j(X) - b_j(X)f_j(X) = q_j(X)Z_{\mathbb{K}}(X)$. The prover commits to $\{f_j\}_{j \in [D]}$ before receiving the batching challenge.
 - (j) $\mathcal{V}$ samples a random $\gamma \leftarrow \mathbb{F}$ and sends it to $\mathcal{P}$.
 - (k) $\mathcal{P}$ combines the already committed polynomials as $\sum_{j=0}^{D-1} \gamma^j (a_j(X) - b_j(X)f_j(X)) = Q_2(X)Z_{\mathbb{K}}(X)$ and commits to Q_2 .
 - (l) $\mathcal{V}$ samples a random $\beta_2 \leftarrow \mathbb{F}$ and sends it to $\mathcal{P}$.
 - (m) Let $G_2 = \{Q_2, f_j\}$ and $Y_2 = \{Q_2(\beta_2), f_j(\beta_2)\}$. $\mathcal{P}$ runs $\pi_2 = \text{PCS.BatchEval}(\text{srs}, G_2, \beta_2)$ and sends (Y_2, π_2) to $\mathcal{V}$.
 - (n) $\mathcal{V}$ checks whether
$$Q_2(\beta_2) \cdot Z_{\mathbb{K}}(\beta_2) \stackrel{?}{=} \sum_{j=0}^{D-1} Z_{\mathbb{H}}(\beta_1) \cdot Z_{\mathbb{H}}(\alpha_1) \cdot Z_{\mathbb{H}}(\alpha_2) \cdot w_j(\beta_1) \cdot \hat{\text{val}}_j(\beta_2) - (\beta_1 - \hat{\text{col}}_j(\beta_2))(\alpha_1 - \hat{\text{row}}_j(\beta_2))(\alpha_2 - \hat{\text{mat}}_j(\beta_2)) \cdot f_j(\beta_2),$$
 and $\sum_{j=0}^{D-1} f_j(0) \cdot |\mathbb{K}| \stackrel{?}{=} \sigma$.
 - (o) Upon successful verification, $\mathcal{V}$ substitutes σ into the Round 1 equation to complete the wire consistency check.
 - **Verify:** Let $C = \{\{w_j\}_{j=0}^{D-1}, Q_1, R, Q_2, \{f_j\}_{j=0}^{D-1}\}$. $\mathcal{V}$ runs:
 - (1) $b_{\pi_1} = \text{PCS.BatchVerify}(\text{srs}, C_1, \beta_1, Y_1, \pi_1)$ where $C_1 = \{\{w_j\}_{j=0}^{D-1}, Q_1, R\}$.
 - (2) $b_{\pi_2} = \text{PCS.BatchVerify}(\text{srs}, C_2, \beta_2, Y_2, \pi_2)$ where $C_2 = \{Q_2, \{f_j\}_{j=0}^{D-1}\}$.
 - (3) Verify the two sumcheck equations from Round 1 and Round 2.
 - (4) If all checks pass, $\mathcal{V}$ outputs $b = 1$; otherwise outputs $b = 0$.

satisfy all gates of the B SIMD subcircuits. It remains to show that these locally satisfying assignments obey the wiring of the padded general circuit.

For $i < D$ and $x \in \mathbb{H}$, define

$$e_i(x) := \sum_{j=0}^{D-1} \sum_{y \in \mathbb{H}} \hat{M}_{ij}(x, y) \hat{w}_j(y) - \hat{w}_i(x),$$

where the padded coordinates are zero. The desired wiring relation

$$\mathbf{M} \vec{w}^T = \vec{w}^T$$

holds exactly when $e_i(x) = 0$ for all i and x .

Suppose otherwise. For every $i < D$, define

$$G_i(A) := \sum_{x \in \mathbb{H}} u_{\mathbb{H}}(A, x) e_i(x),$$

and aggregate these polynomials as

$$\Phi(A_1, A_2) := \sum_{i=0}^{D-1} u_{\mathbb{H}}(A_2, z_i) G_i(A_1).$$

Since the interpolation kernels form scaled Lagrange bases on $\mathbb{H}$, a nonzero wiring-error vector implies that Φ is a nonzero polynomial. Moreover, its degree is at most $D - 1$ in each variable. Hence, for independent uniform challenges $\alpha_1, \alpha_2 \leftarrow \mathbb{F} \setminus \mathbb{H}$,

$$\Pr[\Phi(\alpha_1, \alpha_2) = 0] \leq \frac{2(D-1)}{|\mathbb{F}| - D}. \quad (\text{A.3})$$

Condition on $\Phi(\alpha_1, \alpha_2) \neq 0$. Let $p_1(X)$ denote the polynomial on the left-hand side of the first sumcheck identity in Protocol F.1. The protocol claims

$$p_1(X) = Q_1(X)Z_{\mathbb{H}}(X) + XR(X), \quad \deg Q_1, \deg R \leq D - 2. \quad (\text{A.4})$$

Summing the right-hand side of (A.4) over $X \in \mathbb{H}$ gives zero: the $Z_{\mathbb{H}}$ term vanishes on $\mathbb{H}$, while $XR(X)$ has degree at most $D - 1$ and zero constant coefficient. By construction,

$$\sum_{x \in \mathbb{H}} p_1(x) = \Phi(\alpha_1, \alpha_2) \neq 0.$$

Thus (A.4) is a false polynomial identity.

All polynomials in (A.4) are fixed before $\beta_1 \leftarrow \mathbb{F} \setminus \mathbb{H}$ is sampled, and the difference between its two sides has degree at most $2D - 2$. Therefore,

$$\Pr[(\text{A.4}) \text{ holds at } \beta_1] \leq \frac{2D-2}{|\mathbb{F}| - D}. \quad (\text{A.5})$$

It remains to justify the preprocessed matrix value used in this first check. After $\alpha_1, \alpha_2, \beta_1$ are fixed, define the polynomials a_j, b_j as in Protocol F.1. Their required relations are

$$a_j(X) - b_j(X)f_j(X) = 0 \quad \text{for every } X \in \mathbb{K}, \quad j < D. \quad (\text{A.6})$$

The commitments to the f_j are fixed before the uniform challenge $\gamma \leftarrow \mathbb{F}$ is sampled. If any relation in (A.6) is false, then the corresponding vector of residues modulo $Z_{\mathbb{K}}$ is nonzero. The random linear combination

$$\sum_{j=0}^{D-1} \gamma^j (a_j(X) - b_j(X)f_j(X))$$

can vanish modulo $Z_{\mathbb{K}}$ with probability at most

$$\frac{D-1}{|\mathbb{F}|}. \quad (\text{A.7})$$

Conditioned on non-cancellation, the polynomial identity

$$\sum_{j=0}^{D-1} \gamma^j (a_j(X) - b_j(X)f_j(X)) = Q_2(X)Z_{\mathbb{K}}(X) \quad (\text{A.8})$$

is false and is fixed before β_2 is sampled. Its difference polynomial has degree at most $4B - 4$, and hence its random-point check accepts with probability at most

$$\frac{4B-4}{|\mathbb{F}|}. \quad (\text{A.9})$$

Outside the events in (A.7) and (A.9), all relations in (A.6) hold. Since $\deg f_j < B$ and $\mathbb{K}$ is the size- B evaluation domain,

$$\sum_{k \in \mathbb{K}} f_j(k) = Bf_j(0).$$

Together with PCS extraction and opening consistency, this implies that the value σ used in the first sumcheck is the correct evaluation of the preprocessed sparse matrix. Consequently, the first sumcheck identity can hold only if

$$\Phi(\alpha_1, \alpha_2) = 0,$$

contradicting the conditioned wiring error.

Therefore, outside the stated bad events, the extracted SIMD assignments satisfy both the local subcircuit constraints and the global wiring relation. Reading the assignment in the original circuit order and discarding padded dummy gates yields a witness for the original circuit. Summing the PCS, SIMD, and polynomial-testing failure probabilities proves the theorem. $\square$

G Optimizations of Instantiating AHE

Following previous works, we use Ring-LWE (RLWE)-based schemes (such as BGV [11]) to instantiate the AHE primitive and construct our PCS. Below we present some practical optimizations for our instantiation.

G.1 Inner-product Computation

RLWE-based AHE schemes support SIMD-style plaintext packing, which allows multiple independent messages to be processed in parallel within a single ciphertext. Concretely, the plaintext space is decomposed into multiple independent slots, enabling homomorphic operations to be applied slot-wise with the cost of a single ciphertext operation.

However, in our PCS, the prover's commitment to polynomials involves inner-product computations that aggregate values across slots within a ciphertext. Such cross-slot operations are expensive, as they require a sequence of ciphertext rotations and additions, resulting in significant computational overhead during the online phase. To address this inefficiency, we adopt two alternative methods to reduce the prover's online computation cost.

Inner-product with transposed-encoding [14]. From the inner-product perspective, we encode each term Λ^i (for $i \in [d]$) as a constant polynomial, and the witness vectors $\{\vec{w}_1, \dots, \vec{w}_N\}$ as polynomials $f_i(x) = \sum_{j=1}^N w_j[i] x^{j-1}$, each aggregating the i -th coordinate across all witnesses. A single homomorphic linear combination

$\sum_{i=1}^d f_i(x) \cdot \langle \Lambda^i \rangle$ then evaluates to

$$\sum_{i=1}^d f_i(x) \cdot \langle \Lambda^i \rangle = \sum_{j=1}^N \left\langle \vec{w}_j, \vec{\Lambda} \right\rangle x^{j-1}.$$

In this way, every coefficient directly encodes one inner product $\langle \vec{w}_j, \vec{\Lambda} \rangle$. The prover masks this polynomial with a random polynomial and returns it. In this way, we can implement batched inner-product computation without any ciphertext rotations in the online phase. Note that the ciphertexts corresponding to Λ^i can be obtained either via rotations from a single ciphertext or as pre-generated public ciphertexts. This yields the same communication-computation tradeoff discussed.

- **Rotation-based derivation.** The prover initially receives a single ciphertext $\langle \Lambda \rangle$. Using rotation keys, the prover locally derives all rotated ciphertexts $\langle \Lambda^i \rangle$. This approach minimizes communication overhead, at the cost of requiring rotation operations during the online phase.
- **Pre-generated ciphertexts.** Alternatively, a trusted third party (TTP) generates and publishes all ciphertexts $\langle \Lambda^i \rangle$ as public parameters. In this case, the prover avoids rotation operations entirely, trading increased communication and public parameter size for lower online computational cost.

Inner-product with polynomial reversal. In our AHE scheme, let $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$ denote the polynomial ring modulo $x^N + 1$, where N is a power of two and q is a sufficiently large modulus to support the desired operations.

Given a vector $\vec{\Lambda} = (\Lambda_0, \dots, \Lambda_{n-1})$, we encode it into a plaintext polynomial:

$$P_l(x) = \Lambda_0 + \Lambda_1 x + \Lambda_2 x^2 + \dots + \Lambda_{n-1} x^{n-1} \in \mathcal{R}_q.$$

The ciphertext $\text{Enc}(P_l)$ is published to the prover, who similarly encodes her witnesses $\mathbf{w} = (w_0, w_1, \dots, w_{n-1})$ into a polynomial:

$$P_r(x) = w_0 - w_1 x^{N-1} + w_2 x^{N-2} + \dots + w_n x^{N-n}.$$

The prover then performs a homomorphic multiplication between the ciphertext $\text{Enc}(P_l)$ and the plaintext polynomial P_r , resulting in:

$$\text{Enc}(P_l \cdot P_r) = \text{Enc}(P_l) * P_r,$$

which encrypts the product polynomial $P_l \cdot P_r \bmod (x^N + 1)$. Then the prover returns $\text{Enc}(P_l \cdot P_r)$ to the verifier. Upon decryption using sk , verifier obtains the polynomial:

$$P_{lr}(x) = P_l(x) \cdot P_r(x) \bmod (x^N + 1).$$

The constant term of $P_{lr}(x)$, i.e., the coefficient of x^0 , is equal to the negative inner product $\sum_{k=0}^{N-1} w_k \Lambda^k$, up to a sign flip due to the reduction modulo $x^N + 1$. This arises from the algebraic identity $x^N \equiv -1 \pmod{x^N + 1}$, which causes wrap-around terms of the form $x^k \cdot x^{N-k} = x^N \equiv -1$, thereby mapping high-degree cross terms back to the constant coefficient with reversed sign. Thus, by appropriately aligning the coefficients of P_l and P_r , the homomorphic multiplication followed by modular reduction effectively aggregates all pairwise products $w_k \Lambda^k$ into the constant term (with negation), allowing A to recover the inner product as:

$$\langle \vec{w}, \vec{\Lambda} \rangle = -[x^0] \left(P_l \cdot P_r \bmod (x^N + 1) \right).$$

However, if the verifier were to obtain the full polynomial, all coefficients except the constant term would reveal additional information about the underlying polynomial (beyond the intended inner product). To prevent this leakage, the prover adds a random masking polynomial to the final ciphertext, i.e.,

$$R(x) = r_1 x + r_2 x^2 + \dots + r_{N-1} x^{N-1},$$

where the coefficients $r_1, \dots, r_{N-1}$ are sampled uniformly at random from the plaintext space. This ensures that only the constant term (which encodes the desired inner product) remains meaningful to the verifier, while the higher-degree coefficients are statistically hidden. This technique leverages the cyclic convolution property induced by the quotient ring $\mathbb{Z}_q[x]/(x^N + 1)$ to enable efficient and secure inner-product computation over encrypted vectors.

Under either approach, the prover can efficiently commit to B slot-wise polynomials, requiring only B ciphertext-plaintext multiplications and B ciphertext additions, thereby substantially reducing the online-phase overhead.

G.2 Ciphertexts Packing

When the batch size is smaller than the available number of plaintext slots, many slots remain unused, leading to inefficient ciphertext utilization. To address this issue, we pack multiple witness polynomials into a single plaintext, thereby increasing slot occupancy.

In our succinct general ZK protocol, multiple ciphertexts are generated; however, after decryption, each ciphertext yields a plaintext polynomial in which only the constant term carries meaningful information, while all higher-degree coefficients are unused. We utilize a packed ciphertext technique that extracts these valid messages and repacks them into a single ciphertext. This process relies on ciphertext rotation and slot-wise addition operations to rearrange and consolidate messages. Concretely, we work in the coefficient representation of AHE and apply a tree-like packing protocol that iteratively combines pairs of ciphertexts using negacyclic shifts and Galois automorphisms. In each level, given two ciphertexts encrypting polynomials $a, b \in \mathcal{R}_q$, we first right-shift b by multiplying a power of x ,

$$b'(x) = x^h \cdot b(x) \bmod (x^N + 1),$$

and then form a bilinear combination

$$c(x) = a(x) + b'(x) + \text{Auto}(a(x) - b'(x), \sigma),$$

where $\text{Auto}(\cdot, \sigma)$ denotes a Galois automorphism (implemented via ciphertext rotation). Repeating this pairing-and-combination step over $O(\log n)$ levels folds all input ciphertexts into a single packed ciphertext, in which the valid messages occupy a structured subset of slots at known positions determined by the packing layout.

Although this packing technique significantly reduces the number of ciphertexts, the resulting ciphertext may still contain unused slots. To further reduce ciphertext size, we optionally perform modulus switching and then exploit the known packing layout to zero out coefficients corresponding to unused slots in the ciphertext component, thereby keeping only the coefficients that actually store useful messages.